



Decomposition of Supply Networks into Subnetworks for Reducing Computational Complexity in Mixed-Integer Optimization Problems

Bachelor's Thesis

As part of the examination:

Bachelor of Science (B. Sc.)

in Computer Science

at the
Baden-Württemberg Cooperative State University Karlsruhe

by

Max Katzenberger

Submission Date	31. August 2026
Processing Period	18.05.2026 - 31.08.2026
Matriculation Number, Course	4008573, TINF23B2
Training Company	SAP SE Dietmar-Hopp-Allee 16 69190 Walldorf, Deutschland
Department	SCM Optimizer
Supervisor of the Training Company	Christoph Eckert
Reviewer of the Duale Hochschule	Heinrich Braun

Declaration of Originality

I hereby declare that I have independently written my Bachelor's Thesis with the topic:

> *Decomposition of Supply Networks into Subnetworks for Reducing Computational Complexity in Mixed-Integer Optimization Problems*

in accordance with § 5 of the "Studien- und Prüfungsordnung DHBW Technik" dated September 29, 2017, and have not used any sources or aids other than those specified. The work has not been submitted to any other examining authority before and has not been published.

Furthermore, I confirm that the submitted electronic version corresponds to the printed version.

Karlsruhe, 17. August 2026

Signed: Katzenberger, Max

Abstract

- English -

This is a short abstract to show the formatting and general style of the template

It is possible to have multiple abstracts in different languages

Abstract

- Deutsch -

Dies ist eine kurze Zusammenfassung, um die Formatierung und den allgemeinen Stil der Vorlage zu zeigen

Es ist möglich, mehrere Abstracts in Unterschiedlichen Sprachen zu haben

Table of Contents

Table of Figures ..... **VII**

Table Directory ..... **VIII**

Source Code Directory ..... **IX**

1 Introduction **1**
1.1 Motivation and Problem 1
1.2 Goal and Research Questions 2

2 Theoretical Foundations **3**
2.1 Mixed Integer Programming 3
2.2 SAP Supply Chain Planning 5
2.2.1 Supply Chains as Graphs 5
2.2.2 Mathematical Model Formulation 9

3 Related Work **12**

4 Methodology **15**
4.1 Research Approach 15
4.2 Evaluation Design 16
4.2.1 Metrics 16
4.2.2 Benchmark Scenarios 17

5 Decomposition Framework Design **20**
5.1 Graph-based Problem Modelling 20
5.2 Decomposition 21
5.3 Two-Phase Solve Approach 27
5.4 Link Node Heuristics 30
5.4.1 Production Cost Aggregation 31
5.4.2 Lead Time Aggregation 32
5.4.3 Max Flow Aggregation 32
5.4.4 Non-Delivery Cost Aggregation 33
5.5 Inter Subnet Constraint Handling 34
5.5.1 Equal Split 34
5.5.2 First-Come First-Serve 35
5.5.3 LP-Relaxation 35
5.6 Solution Combination 36
5.7 Runtime Allocation 37

6 Implementation **40**
6.1 System Architecture 40
6.2 Data Handling and Technologies 42
6.3 Solution Verification 44

7 Evaluation **46**
7.1 Experimental Setup 46
7.2 Scenario Comparison 47

7.3 Subnetwork Granularity 47
7.4 Constraint Handling Modes 48
7.5 Heuristics 48
7.6 Summary 48
8 Discussion 49
9 Conclusion 50
Bibliography ..... a
Index of Appendices ..... A

List of Abbreviations

***BOM* – Bill of Materials** 5, 6, 10, 31, 33, 34

***DAG* – Directed Acyclic Graph** 23, B

***IBP* – Integrated Business Planning** 1, 3, 5, 9, 16, 17, 41

***LP* – Linear Programming** 3, 4, 10

***MILP* – Mixed-Integer Linear Programming** 1, 3, 4, 5, 9, 10, 12, 13, 15, 16, 20, 30, 37, 38, 44, D

***NP* – Nondeterministic Polynomial Time** 1

***PoC* – Proof-of-Concept** 40

***SCC* – Strongly Connected Component** 23

Table of Figures

Figure 1 Example of a supply chain graph with different node types. Source: Internal visualization tool. 8
Figure 2 Decomposition of a supply chain graph into three subnetworks with link nodes and virtual procurements. Source: Internal visualization tool. 22
Figure 3 Example stage classification of nodes. Source: Internal visualization tool. . 24
Figure 4 Example meta graph M for three subnetworks 28
Figure 5 System architecture of the decomposition framework showing the orchestration layer and solver service interaction 40

Table Directory

Table 1 Benchmark scenario characteristics 18
Table 2 Supply chain stage classification 23
Table 3 Runtime budgets for benchmark scenarios 47

Source Code Directory

Listing 1 Retain-if configuration for the production source item table 43

1 Introduction

1.1 Motivation and Problem

Modern supply chain networks have grown increasingly complex, spanning multiple continents and involving thousands of suppliers, production facilities, warehouses, and customers [1]. Managing these networks requires sophisticated optimization systems that determine optimal production quantities, transportation routes, and inventory levels across the entire network. These decisions are typically formulated as *Mixed-Integer Linear Programming (MILP)* problems, which simultaneously optimize continuous variables (e.g., production quantities) and discrete variables (e.g., production lot decisions) [2].

However, the computational complexity of *MILP* problems poses significant challenges for enterprise-scale supply chains. *MILP* is *Nondeterministic Polynomial Time (NP)* - hard in general [3], meaning that solving time can grow exponentially with problem size. Real-world scenarios can involve networks with over one million nodes and optimization problems with more than ten million decision variables. Solving such large-scale problems using traditional monolithic approaches often leads to excessive computation times, sometimes exceeding 14 hours or fails entirely due to memory constraints. In many cases, no feasible solution can be found within acceptable time limits, rendering the optimization system impractical for operational use.

This scalability bottleneck represents a critical limitation for supply chain planning systems such as SAP *Integrated Business Planning (IBP)* [4]. As businesses continue to expand their global operations and supply chain networks grow in complexity, the need for efficient optimization approaches becomes increasingly urgent. The challenge lies in finding methods that can handle the scale of modern supply networks while maintaining solution quality and computational tractability.

A promising approach to address this challenge is decomposition: breaking down the large optimization problem into smaller, more manageable subproblems that can be solved independently and then combined into a global solution [5]. Graph-based decomposition strategies are particularly well-suited for supply chain networks, as these networks naturally exhibit hierarchical structures—from raw material suppliers through manufacturing stages to end customers—that can be exploited for partitioning.

1.2 Goal and Research Questions

The goal of this thesis is to develop and evaluate a decomposition strategy for supply chain network optimization problems. Specifically, this work aims to reduce computational complexity by partitioning large supply networks into smaller subnetworks, representing a part of the supply chain, solving each subnetwork independently, and combining the partial solutions into a feasible global solution.

This thesis addresses the following research questions:

1. **RQ1: How can supply chain networks be decomposed into subnetworks while preserving solution feasibility?**
   This question investigates graph-based decomposition strategies that partition the network while maintaining the structural and logical dependencies necessary for a valid combined solution.
2. **RQ2: How does decomposition affect solution quality compared to computational performance?**
   This question examines if decomposition can impose advantages in runtime usage while preserving the solution quality of monolithic solve approaches.
3. **RQ3: Can decomposition enable solving previously intractable problems?**
   This question evaluates whether the proposed approach can successfully solve large-scale scenarios that would otherwise exceed the computational resource limits of runtime or memory.

2 Theoretical Foundations

This chapter establishes the theoretical foundations underlying the decomposition approach developed in this thesis. Beginning with *MILP*, introducing its formal definition, computational complexity, and motivates the need for decomposition approaches. The chapter then examines how SAP *IBP* Supply Planning models supply chains as directed graphs and transforms them into mathematical optimization problems.

2.1 Mixed Integer Programming

MILP is a fundamental optimization paradigm that extends *Linear Programming (LP)* by requiring some variables to take integer values. A Mixed-Integer Program is an optimization problem of the following form [2]:

$$\begin{aligned} \min \quad & c^T x \\ \text{subject to} \quad & Ax \leq b \\ & x_i \in \mathbb{Z} \quad \forall i \in I \\ & x_i \in \mathbb{R} \quad \forall i \notin I \\ & x \geq 0 \end{aligned} \tag{1.}$$

where $c \in \mathbb{R}^n$ is the objective coefficient vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, $b \in \mathbb{R}^m$ is the right-hand side vector, and $I \subseteq \{1, ..., n\}$ is the index set of variables constrained to integer values.

Several important special cases arise from this general formulation:

- **Pure Integer Program**: When $I = \{1, ..., n\}$, all variables must be integer-valued.
- **Binary Program**: When $x_i \in \{0, 1\}$ for all $i \in I$, variables are restricted to binary values.
- **Linear Program**: When $I = \emptyset$, no integrality constraints exist, yielding a standard LP.

The computational complexity of the solving of a *MILP* places it in the class of NP-hard problems [3]. The fundamental difficulty arises from the integrality constraints. While the feasible region of a linear program is a convex polyhedron [6], requiring even one variable to take integer values transforms the feasible region into a discrete, non-convex set, making the optimization problem NP-hard in general [3].

The source of this hardness lies in the combinatorial explosion of possible integer assignments. If each integer variable is restricted to a finite domain D_i, the search space contains $\prod_i |D_i|$ possible assignments before accounting for the linear constraints.

This exponential complexity has direct implications for solution algorithms. Unlike *LP*, where the simplex method or interior point methods find optimal solutions in polynomial time, no known algorithm solves general *MILPs* in polynomial time [6].

Every MILP has an associated LP relaxation. The LP relaxation is obtained by replacing the variables with integer domains with continuous domains, allowing all integer-constrained variables to take fractional values [2]:

$$z_{\text{LP}} = \min\{c^T x : Ax \leq b, x \in \mathbb{R}^n, x \geq 0\} \tag{2.}$$

This relaxation possesses several important properties. For minimization problems, the LP relaxation provides a lower bound on the optimal MILP objective value: $z_{\text{LP}} \leq z_{\text{MILP}}$. This relationship holds because the feasible region of the LP relaxation contains all integer-feasible solutions [2]. The quality of the relaxation is measured by the integrality gap:

$$\text{gap} = \frac{z_{\text{MILP}} - z_{\text{LP}}}{|z_{\text{MILP}}|} \tag{3.}$$

A small integrality gap indicates that the LP relaxation closely approximates the MILP, making it a "tight" relaxation. While solving general MILPs is NP-hard, linear programs can be solved in polynomial time [6], making LP relaxation relevant for bounding and approximation purposes.

The exponential worst-case complexity motivates the use of decomposition approaches. The general idea is to exploit the structure of a large optimization problem by dividing it into smaller subproblems, thereby reducing the complexity of the search space. Picture a *MILP* with n binary variables, the number of possible assignments can grow exponentially as 2^n. If the problem can be decomposed into k smaller subproblems of size $\frac{n}{k}$, the corresponding search space is reduced to approximately $k \cdot 2^{\frac{n}{k}}$, illustrating the potential computational benefits of decomposition.

This scaling behavior makes decomposition particularly attractive for large-scale optimization. However, decomposition approaches introduce additional challenges. When subproblems are coupled through shared variables or constraints, solving them independently may no longer preserve global optimality. While exact decomposition methods can maintain optimality guarantees, they often require additional coordination and

iterative procedures [7], [8]. In practice, large-scale MILPs are frequently not solved to proven optimality due to computational limits, but rather until a satisfactory solution is obtained within a given time budget. Therefore, approximate decomposition approaches may trade optimality guarantees for computational efficiency, aiming to obtain high-quality solutions within reasonable time limits [9].

Modern MILP solvers use branch-and-bound algorithms enhanced with presolve, cutting planes, and primal heuristics, achieving quick solution speeds for big *MILPs* [9]. However, very large and complex *MILPs* will still require problem-specific formulations or decomposition approaches to achieve better solution times.

2.2 SAP Supply Chain Planning

SAP *IBP* Supply Planning provides an enterprise platform for supply chain optimization that transforms real-world supply networks into mathematical models [4]. This section introduces the conceptual foundations of how SAP Supply Planning represents supply chains and formulates optimization problems, establishing the context for the decomposition approaches developed in this thesis.

2.2.1 Supply Chains as Graphs

Modern supply chains consist of connected products, operations and customers that can be naturally represented as directed graphs [1]. SAP *IBP* Supply Planning leverages this graph-theoretic perspective to model supply networks, where vertices represent operational entities and edges capture material flows and dependencies.

The foundation of Supply Planning is a relational data model organized into several categories of input tables [4]. Example categories are given by:[1]

- **Location-Product Tables** define which products exist at which locations in the supply network, establishing the basic inventory points.
- **Customer-Product Tables** represent a product at a customer, modeling possible demand points where customer orders may occur.
- **Production Tables** define manufacturing operations through production rules that encode *Bill of Materials (BOM)* relationships, specifying which input materials and in what quantities are required to produce each output product.

[1] The tables vary based upon interface, version and release, but can generally be categorized by those example.

- **Transport Tables** describe logistics lanes between locations, capturing lead times, capacities, and transportation costs for moving materials through the network.
- **Customer-Transport Tables** define delivery lanes from locations to customers, specifying how finished products reach demand points.
- **Procurement Tables** model external supply sources, defining available quantities, costs, and constraints for purchasing materials from suppliers.
- **Resource Tables** capture capacity constraints for production equipment, storage facilities, and transportation vehicles that limit operational throughput.

Each table contains time-indexed key figures that specify period-by-period values for quantities, costs, and capacity limits across the planning horizon.

From these relational tables, a directed graph $G = (V, E)$ can be constructed to capture the supply chain topology. This graph abstraction makes explicit the relationships between entities that are implicit in the relational tables, enabling structured partitioning for decomposition approaches. By dividing the graph into subgraphs, it can be directly determined which portions of the input data belong to each subproblem. The vertex set V comprises several node types that represent distinct operational entities:

- **Location-Product nodes** represent the existence of a specific product at a specific location.
- **Production nodes** represent production operations that transform input materials into output products according to *BOM* specifications at a specific location.
- **Transport nodes** represent logistics operations that move materials between locations.
- **Customer-Product nodes** represent demand points where products must be delivered to customers.
- **Customer-Transport nodes** represent delivery operations that move finished products from locations to customers.
- **Procurement nodes** represent external supply sources from which materials can be obtained.
- **Resource nodes** represent capacity-constrained assets such as production equipment, storage facilities, and transportation vehicles.

The edge set E captures the material flow and dependencies of resources on operations. Figure 1 illustrates an example of such a supply chain graph with material flow from top to bottom.

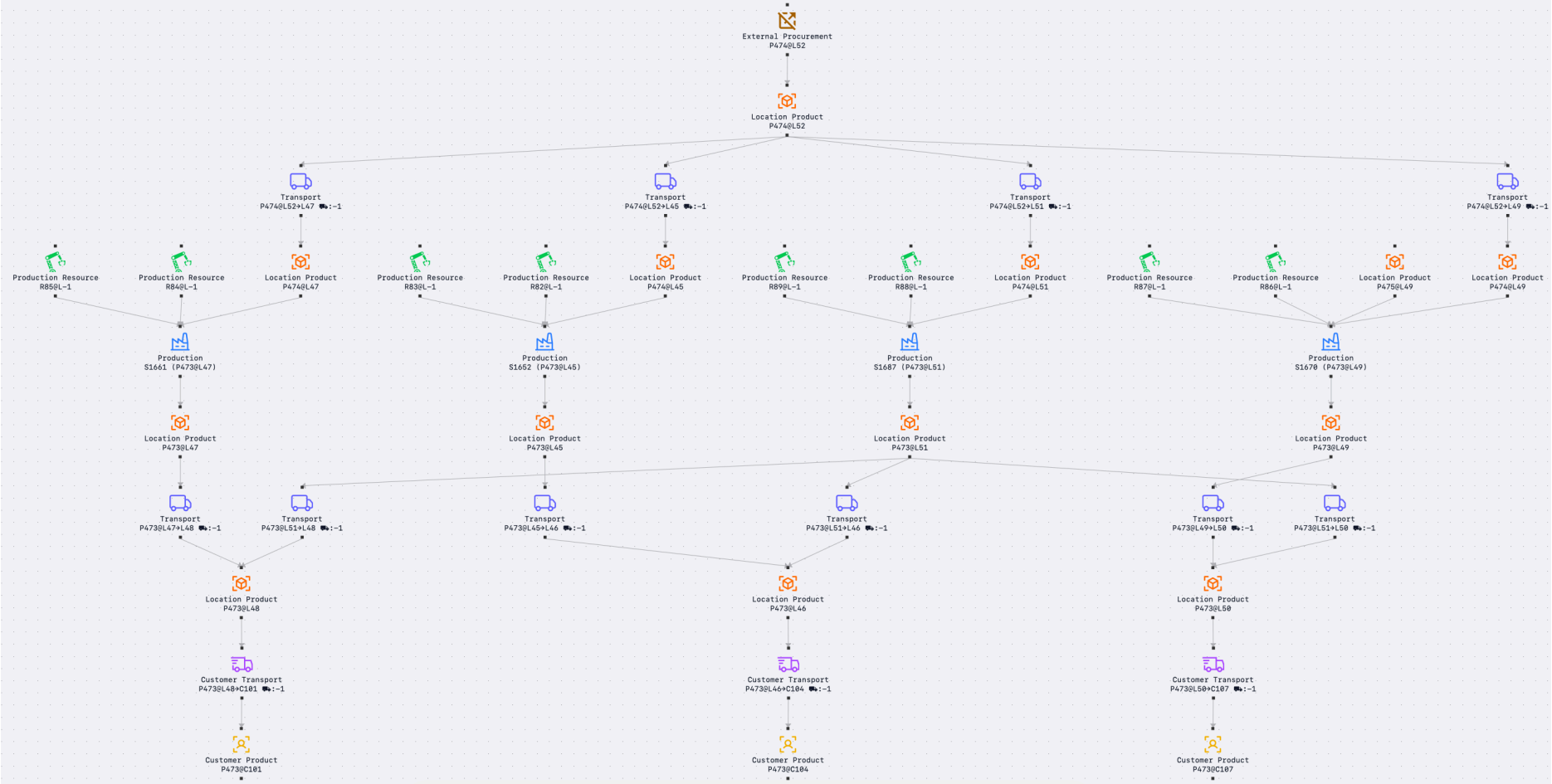


Figure 1: Example of a supply chain graph with different node types. Source: Internal visualization tool.

For the purposes of this thesis, the standard optimization pipeline in SAP *IBP* is modeled as:

$$\text{Input Tables} \rightarrow \text{Object-oriented Model} \rightarrow \text{MILP Model} \rightarrow \text{Solver} \rightarrow \text{Solution} \quad (4.)$$

The relational input tables are used to build the object-oriented model which is then transformed into a *MILP* formulation. The *MILP* is then submitted to an external solver such as Gurobi or SCIP/SoPlex [10], [11]. Finally, the solver's solution is interpreted back into operations, like how much to produce, what to transport, and when to procure materials.

Separately, the supply chain graph $G = (V, E)$ serves as an auxiliary abstraction built from the same input tables. This graph representation provides structural insight useful for analysis and, as explored in this thesis, for decomposition strategies. When any form of structural decomposition is applied, the graph is constructed from the input tables, partitioned into subgraphs, and these subgraphs guide how to filter or modify the original input data to create subproblems (see Section 6.2). Each subproblem's modified input data is then transformed into a *MILP* and solved.

This pipeline architecture enables SAP *IBP* to leverage external *MILP* solvers while maintaining a domain-specific interface for supply chain planners [4].

2.2.2 Mathematical Model Formulation

SAP *IBP* formulates the supply chain planning problem as a *MILP* containing variables, constraints and costs [12], [13].

The *MILP* formulation introduces decision variables that correspond to operational quantities across all time buckets in the planning horizon [12]. Let $T = \{1, 2, ..., n\}$ be the set of planning buckets. An example of the primary decision variables are:

- $\text{VO}_{o,s}(t)$ – the number of production lots of model o started at location s in period t
- $\text{VT}_{e,p}(t)$ – the quantity of material p transported on edge e in period t
- $\text{VS}_{s,p}(t)$ – the inventory level of material p at location s at the end of period t
- $\text{VL}_{d}(t)$ – the quantity delivered for demand d in period t

Various business requirements can introduce integer or binary variables into the formulation. For example, when lot sizes are active, production can only occur in discrete multiples of the lot size L_o, requiring $\text{VO}_{o,s}(t) \in \mathbb{Z}$. The output of the operation is then

given by $L_o \cdot \mathrm{VO}_{o,s}(t)$. Such integrality requirements make the problem a *MILP* rather than a pure *LP*.

The objective function minimizes total supply chain costs, balancing operational costs against delivery penalties [2]:

$$\begin{aligned}\min \sum_{t \in T} \Bigg[& \sum_{s \in S} \sum_{o \in O_s} \mathrm{CO}_{o,s} \cdot \mathrm{VO}_{o,s}(t) + \sum_{s \in S} \sum_{p \in \mathrm{PS}_s} \mathrm{CS}_{s,p} \cdot \mathrm{VS}_{s,p}(t) \\ & + \sum_{e \in E} \sum_{p \in \mathrm{PE}_e} \mathrm{CT}_{e,p} \cdot \mathrm{VT}_{e,p}(t) \\ & + \sum_{d \in D} \mathrm{CD}_d \cdot \left(\mathrm{DQ}_d - \sum_t \mathrm{VL}_d(t) \right) \Bigg]\end{aligned} \tag{5.}$$

where $\mathrm{CO}_{o,s}$ denotes production costs per unit of production model o at location s; $\mathrm{CS}_{s,p}$ represents inventory holding costs; $\mathrm{CT}_{e,p}$ captures transportation costs per unit on edge e and CD_d is the penalty for each unit of unfulfilled demand d.

The non-delivery penalty term ensures that the model prefers fulfilling demand when feasible, with the penalty magnitude reflecting the business cost of missed deliveries.

The *MILP* formulation enforces physical and operational constraints that link the decision variables. The following presents the constraints most relevant for understanding the decomposition approach developed in this thesis:

Location-Product Constraints. For each location-product node, the inventory balance ensures that stock changes reflect all inflows and outflows:

$$\begin{aligned}\mathrm{VS}_{s,p}(t) = \mathrm{VS}_{s,p}(t-1) & + \sum_{o \in O_s} \mathrm{PR}_{o,s,p} \cdot \mathrm{VO}_{o,s}(t) \\ & + \sum_{e \in E_s} + \mathrm{VT}_{e,p}(t) - \sum_{e \in E^s} \mathrm{VT}_{e,p}(t) - \mathrm{VL}_d(t)\end{aligned} \tag{6.}$$

where $\mathrm{PR}_{o,s,p}$ is the *BOM* coefficient representing the quantity of material p produced (positive) or consumed (negative) by production model o at location s. Location-product nodes contribute inventory holding costs $\mathrm{CS}_{s,p} \cdot \mathrm{VS}_{s,p}(t)$ to the objective.

Resource Constraints. Resource nodes impose shared capacity limits across multiple operations. Three types of resources exist: production resources constrain production operations, transport resources constrain transport operations, and storage resources constrain inventory levels at locations. All three follow the same constraint structure. The resource consumption is computed directly in the capacity constraint by sum-

ming used quantities multiplied by resource requirement coefficients. For production resources:

$$\sum_{o \in O_{s_r}} \mathrm{RR}_{o,r} \cdot \mathrm{VO}_{o,s}(t) \leq \mathrm{RC}_r(t) \tag{7.}$$

where $\mathrm{RR}_{o,r}$ is the resource requirement for production model o on resource r, and $\mathrm{RC}_r(t)$ is the capacity of resource r. Transport and storage resources follow analogously, with their respective variables VT and VS replacing VO. This constraint couples multiple operational decisions through shared capacity. Resource nodes do not directly contribute to the objective but constrain feasible solutions.

Non-Negativity. All quantity variables must be non-negative:

$$\mathrm{VO}_{o,s}(t), \mathrm{VT}_{e,p}(t), \mathrm{VS}_{s,p}(t), \mathrm{VL}_d(t) \geq 0 \tag{8.}$$

The complete mathematical formulation is not presented here, as it is beyond the scope of this thesis and would not contribute to the discussion of the proposed decomposition approach anyways.

3 Related Work

Mathematical decomposition techniques for mixed-integer linear programming are well established within the optimization community. Methods such as Benders decomposition [14], Lagrangian relaxation [15], [16], [17], [18], and Dantzig-Wolfe decomposition [19] operate at the solver level by exploiting algebraic structure in the constraint matrix [20], [21]. These techniques are domain-agnostic, applying generically to any *MILP* formulation without leveraging knowledge about what the variables and constraints represent. The more interesting research question for supply chain optimization is how to decompose problems at the domain level, exploiting the network structure that defines how materials flow from suppliers through production stages to customers.

SAP Integrated Business Planning incorporates heuristic decomposition approaches to manage the computational complexity of supply network planning [22], [23]. Sandner [12] provides the foundational reference for SAP's decomposition strategies. These methods build upon classical decomposition theory [24] but adapt it for the practical constraints of mixed-integer formulations where exact methods cannot be directly applied due to integrality requirements [25], [26]. Four primary decomposition strategies are implemented: priority decomposition, product decomposition, time decomposition, and resource decomposition. Priority decomposition partitions demands by priority class, solving subproblems in descending order to ensure high-priority demands claim capacities first. Time decomposition partitions along the temporal dimension, taking subsets of time buckets successively into the planning window while aggregating future periods into coarser buckets [27]. This rolling horizon approach relates conceptually to the multi-period planning structures studied in operations research [28]. Product decomposition partitions along the material dimension, drawing on concepts from decentralized planning [29]. Resource decomposition partitions the problem based on subsets of resources, grouping production models and materials that depend on the selected resources [12]. A preprocessing phase analyzes the material dependency graph to find connected components, and resources are allocated across subproblems using a first-come-first-served strategy. The key structural characteristic is that product decomposition cuts horizontally through the supply network, following the direction of material flow [12].

The more similar approach to this thesis is cutting through the material flow and solving different stages independently. Stage-wise decomposition for supply chains was pioneered by Clark and Scarf [30], who introduced echelon-based decomposition for

stochastic inventory systems, solving each stage separately through a single backward pass from downstream to upstream. Coupling between stages is handled through penalty terms in the cost function, achieving exact optimality for serial structures. This foundational work was extended in various directions [31], [32], [33], [34], but these results apply to stochastic dynamic programming for pure inventory problems rather than *MILP*.

Multi-echelon decomposition approaches that partition networks along functional or organizational boundaries face the challenge of coordinating the resulting subproblems [35]. Barbarosoglu and Ozgur [7] decomposed production-distribution planning into a two-level hierarchy, with coupling handled through Lagrangian relaxation where multipliers serve as price signals updated via subgradient optimization. This approach requires multiple iterations and provides no direct feasibility guarantee. The work of Lidestam and Ronnqvist [8] provides a particularly instructive case study. They decompose a supply chain into upstream procurement-production and downstream distribution subproblems, with flow variables duplicated at the boundary and Lagrangian multipliers penalizing discrepancies. Their computational experiments reveal the limitation that even after ten iterations, solutions frequently remain infeasible or exhibit gaps ranging from approximately one percent to over seventy percent, with several instances producing no feasible solution at all.

The dominant coordination mechanism in the literature is Lagrangian-based price coordination [7], [8], [17], [18], [27]. Lagrange multipliers are iteratively updated through subgradient optimization or bundle methods. These methods share common limitations as subproblems must be solved multiple times as coordination signals converge, and solutions to relaxed subproblems may violate coupling constraints that were relaxed, requiring heuristic repair procedures whose success cannot be assured [21].

Existing domain-level decomposition approaches thus share the limitations that they require iterative coordination between subproblems, and they provide no inherent feasibility guarantee. This thesis addresses this gap by proposing a graph-structural decomposition that exploits supply chain topology to coordinate subproblems through quantity-based coupling in one up and down pass. Rather than relying on price signals that require iterative adjustment, the approach uses hard flow constraints to maintain feasibility and objective function penalties to drive towards better solutions. The bottom-up phase propagates demand requirements while the top-down phase injects actual supply quantities, which means a single up and down pass provides guarenteed

feasibilty rather than recovering it through post-hoc repair. This eliminates the iteration requirement while guaranteeing that the composed solution satisfies all constraints across subnetwork boundaries.

4 Methodology

This chapter describes the research methodology employed in this thesis and the evaluation design used to assess the proposed decomposition approach. The methodology follows the Design Science Research paradigm, which is well-suited for developing and evaluating artifacts that address practical problems in information systems.

4.1 Research Approach

This thesis follows the Design Science Research (DSR) methodology, a research paradigm focused on creating and evaluating artifacts to solve identified organizational problems [36]. DSR is particularly appropriate for this work because the primary goal is to develop a practical solution that addresses the computational challenges of large-scale supply chain optimization.

The artifact developed in this thesis is a decomposition framework for supply chain networks modeled as *MILP* problems. This framework partitions complex supply networks into smaller subnetworks that can be solved independently, with the goal of reducing computational complexity while maintaining solution feasibility. The key components of this artifact are:

- A graph-based representation of supply chain networks
- A decomposition strategy for partitioning the network
- A two-phase solving approach (bottom-up request phase, top-down correction phase)
- Virtual link constraints for connecting subnetworks
- Heuristics for aggregating information across subnet boundaries
- Constraint handling mechanisms for shared capacity constraints
- Solution combination and verification process

The DSR process in this thesis follows four main phases:

1. **Problem Identification and Motivation:** The limitations of monolithic *MILP* solving for enterprise-scale supply chains were identified and analyzed. This reasoning shows the practical relevance and necessity of a decomposition approach.
2. **Objectives Definition:** Clear objectives were defined based on the research questions. Preserving solution feasibility (RQ1), showing the quality-performance improvement (RQ2), and enabling previously unsolvable problems to be solved (RQ3).

3. **Design and Development:** The decomposition framework is designed and implemented, including graph-based modeling, the decomposition strategy, and the two-phase solving approach.
4. **Evaluation:** The artifact is evaluated against defined metrics using real-world benchmark scenarios, comparing performance against a monolithic baseline.

4.2 Evaluation Design

The evaluation compares the proposed decomposition approach with a monolithic baseline, which represents the standard method used in SAP *IBP* for supply chain optimization. In the baseline approach, the entire supply network is formulated as a single *MILP* problem and solved without decomposition. For each scenario, the same optimization problem is solved once using the monolithic formulation and once using the decomposition approach, with both runs subject to the same time limit and using the same solver with identical parameter settings. This controlled setup isolates the effect of decomposition from other variables, allowing direct attribution of performance differences to the decomposition approach itself.

4.2.1 Metrics

The evaluation uses the two primary metrics that represent the fundamental trade-off in optimization [2].

Runtime measures the solver time required to solve the optimization problem, excluding preprocessing and decomposition overhead. This focus on pure solver time is deliberate, since the primary research interest lies in understanding how decomposition affects the complexity of the underlying optimization problems themselves. By measuring only the time spent in the *MILP* solver, the evaluation captures the impact of partitioning large problems into smaller subproblems. Additionally, the current implementation introduces overhead due to architectural design decisions that would not be present in a production integration within SAP *IBP* (see Section 6), where decomposition logic could be tightly integrated with the existing optimization pipeline. Comparing total runtimes would confuse this implementation-specific overhead with the actual optimization benefits, obscuring the true potential of the decomposition approach.

Solution Quality is measured by the objective value of the resulting solution. In supply chain optimization, the objective function minimizes the total costs. Lower objective values indicate better solutions.

To ensure fair comparison between the decomposition approach and the monolithic baseline, both runs are given the same runtime budget, which corresponds to the time limit used by the customer in production. The solution quality achieved within this budget is then compared. If one approach solves the problem to proven optimality before the time limit, the runtime budget for the other approach is reduced to match this shorter time, ensuring a fair comparison at the point of optimality. If a run fails to find any feasible solution within the given time, its solution quality is reported as infeasible. In such cases, any feasible solution found by the other approach represents a clear improvement, regardless of its objective value.

4.2.2 Benchmark Scenarios

The evaluation uses ten real customer scenarios from SAP *IBP*, deliberately selected to represent diverse network structures and complexity characteristics. These scenarios are anonymized production data that show the variety of supply chain configurations in practice. The selection aims to identify which network characteristics make decomposition beneficial and which scenarios may not benefit from the approach. Some scenario characterisitics are shown in Table 1.

Attribute	**S1**	**S2**	**S3**	**S11**	**S13**
Graph Attributes:					
Total nodes	116432	38536	976	21097	1047357
Loc-Prods	54981	18145	203	7609	185608
Productions	13868	10088	119	1433	55004
Transports	36606	6701	262	7879	796557
Customer-Prods	5797	2717	149	4112	9621
Procurements	5051	402	236	4	567
Resources	129	483	7	60	0
Monolithic Model:					
Variables	1261061	6963608	66361	150053	212315069
Discrete Variables	589508	125651	6644	44344	4299160
Linear constraints	446301	3563344	33117	63010	54179932

Attribute	**S14**	**S15**	**S17**	**S18**	**S19**
Graph Attributes:					
Total nodes	1042863	234061	481061	116432	234039
Loc-Prods	275932	26862	70212	54981	26862
Productions	21472	1666	40667	13868	1666
Transports	525009	132360	199813	36606	132357
Customer-Prods	219890	72656	166006	5797	72656
Procurements	165	474	2327	5051	455
Resources	395	43	2036	129	43
Monolithic Model:					
Variables	10121093	1822553	—	1261061	6673859
Discrete Variables	2240208	43326	—	589508	145094
Linear constraints	3418043	607744	—	446301	2130711

Table 1: Benchmark scenario characteristics

Beyond the fundamental comparison between decomposition and monolithic solving, the evaluation also investigates how different configuration parameters affect decomposition performance. The decomposition framework developed in this thesis offers three configurable aspects:

- the number of subnetworks,

- the handling of shared constraints across subnetworks,
- four heuristics for information aggregation.

These configuration options are described in detail in Section 5. By systematically varying these parameters across the benchmark scenarios, the evaluation aims to identify not only whether decomposition is beneficial, but also which configurations work best for different types of supply chain structures.

5 Decomposition Framework Design

The starting point of the framework is a supply chain defined by master data tables, which can be directly transformed into a *MILP* formulation and solved monolithically, i.e., as a single unified problem, using the standard optimization service. To enable decomposition, we construct a directed graph $G = (V, E)$ as a structural abstraction from these tables, as described in Section 2.2.1. This chapter addresses the design of a decomposition framework that partitions this graph into smaller subgraphs, uses these subgraphs to derive the modified subproblems for each subnetwork, coordinates the optimization of the resulting subproblems, and combines their solutions into a feasible solution for the original problem.

The decomposition framework consists of three interconnected components:

1. **Graph Partitioning**: The graph G must first be partitioned into a set of subgraphs $\{G_1, G_2, ..., G_k\}$ based on the topological structure of the network. The partitioning should exploit the structure of the supply chain, such as the material flow from procurement through production to distribution.
2. **Subproblem Coordination**: Each subgraph G_i defines a subset of the supply chain, and the corresponding input data is extracted to construct a *MILP* subproblem. A coordination mechanism is required to determine the order in which subproblems are solved, the information exchanged across partition boundaries, and how intermediate results are incorporated into subsequent optimization steps.
3. **Constraint Preservation**: Constraints in the original formulation may span multiple subgraphs and therefore cannot always be assigned to a single subproblem. These constraints must be distributed or reformulated so that the combined solution remains feasible with respect to the original *MILP* formulation.

5.1 Graph-based Problem Modelling

The decomposition framework operates on a directed graph $G = (V, E)$ that represents the supply chain network. This graph-based representation captures the flow of materials through the network, defined by the master data tables described in Section 2.2.1.

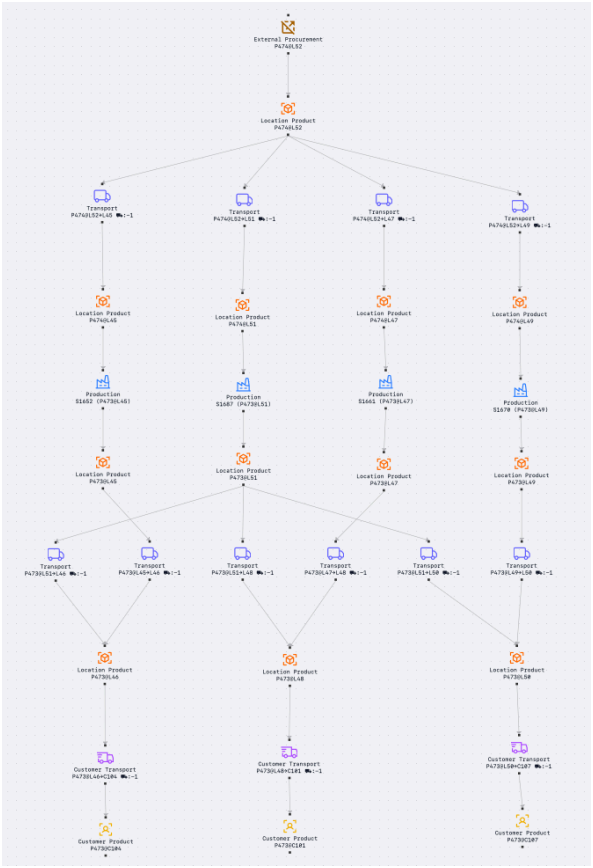
The graph is directed to reflect the natural flow of materials in supply chains. Unlike simple tree structures, supply chain graphs are multi-source, multi-sink and can contain cycles. Cycles arise in practical scenarios through product substitution relationships,

where alternative products can fulfill the same demand, and through closed-loop supply chains involving recycling or rework processes [37]. The presence of cycles has important implications for the decomposition approach discussed in subsequent sections.

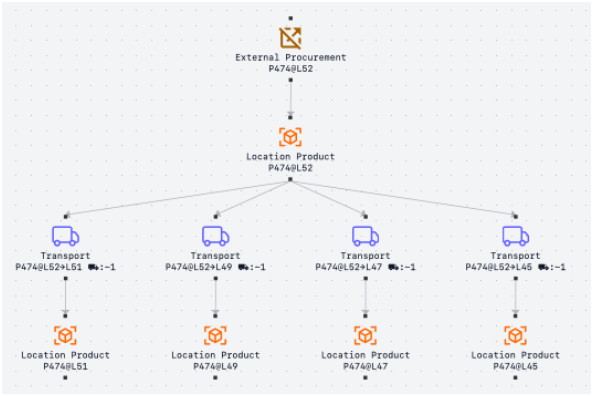
Each node $v \in V$ represents an operational entity in the supply chain and carries a type attribute corresponding to the node categories introduced in Section 2.2.1. Edges $e \in E$ represent material flow relationships and resource dependencies.

5.2 Decomposition

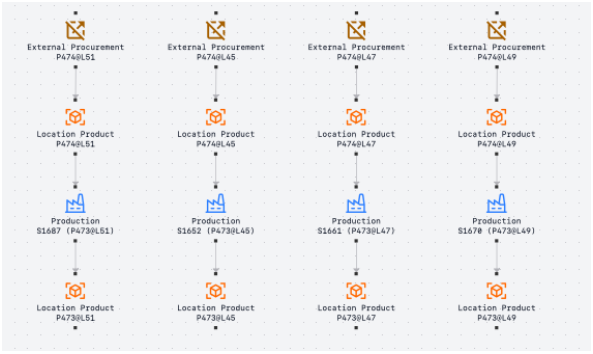
The decomposition partitions the supply chain graph $G = (V, E)$ into k disjoint subgraphs $\{G_1, G_2, ..., G_k\}$ based on the position of nodes within the supply chain. The goal is to create subproblems that can be solved sequentially, while each subnetwork represents an actual stage in a supply chain, e.g. Procurement, Production or Distribution. Figure 2 shows the target state of this decomposition process. The figure shows the several concepts that are now formally defined throughout this section.



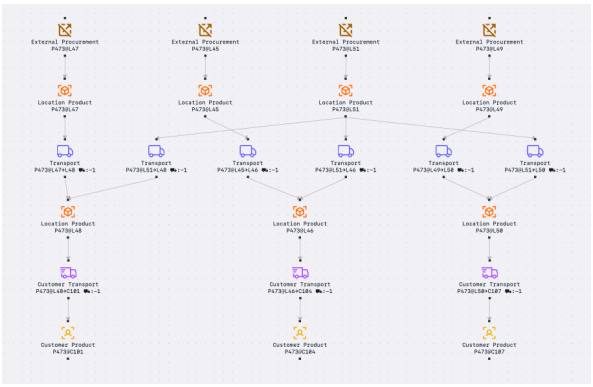
(a) Original supply chain graph



(b) Subnetwork 1: Procurement



(c) Subnetwork 2: Production



(d) Subnetwork 3: Distribution

Figure 2: Decomposition of a supply chain graph into three subnetworks with link nodes and virtual procurements. Source: Internal visualization tool.

Cycles

The stage assignment and partitioning relies on the relative position of nodes and well-defined up- and downstream relationships. Cyclic dependencies invalidate this assumption, making the assignment ambiguous. Furthermore, partitioning a cycle across multiple subnetworks introduces circular dependencies between them, preventing the

subnetworks from being ordered for sequential optimization. Consequently, cycles must remain entirely within a single subnetwork and may not be split during partitioning.

To achieve this, the decomposition operates on the condensation $G_C = (V_C, E_C)$ of the network rather than on G directly. The condensation is constructed by first identifying all *Strongly Connected Components (SCCs)*, then contracting each *SCC* into a single meta-node. An edge exists between two meta-nodes in G_C if and only if an edge exists between their corresponding components in G. Since all cycles are absorbed into the contracted nodes, the condensation is always a *Directed Acyclic Graph (DAG)*, enabling topological traversal. The stage classification therefore assigns stages to meta-nodes (SCCs) rather than individual nodes, and all nodes within an *SCC* inherit the same stage from their containing meta-node.

Classification

Each node $v \in V_C$ is assigned a supply chain stage $s(v) \in \{1, 2, 3, 4, 5\}$ based on its relative position inside the network. The five stages reflect the typical phases of material flow in supply chains: sourcing from suppliers, pre-processing, production/transformation, post-processing of finished goods, and distribution to customers. This classification provides the finest meaningful granularity and coarser decompositions can be achieved by grouping stages together, as described in the following section. Table 2 summarizes the five stages and their criteria and Figure 3 shows and example classification of nodes.

Stage	**Name**	**Criterion**
1	Sourced	Is procurement or input from procurement
2	Pre-Production	Upstream of all production, no direct sourcing
3	Intermediate	Contains production or lies between production stages
4	Finished	Downstream of all production, no customer connection
5	Distribution	Is customer node or output to customer node

Table 2: Supply chain stage classification

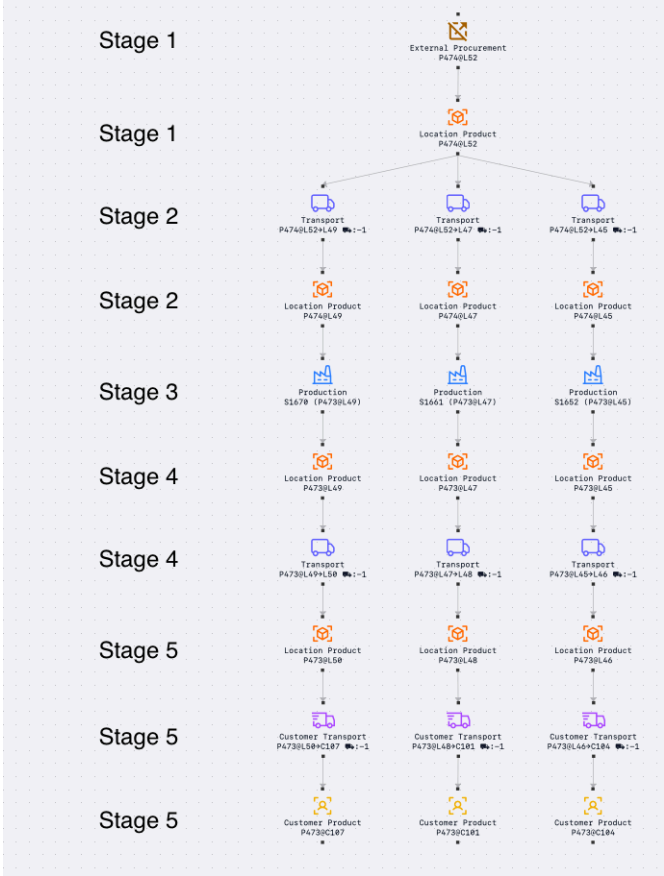


Figure 3: Example stage classification of nodes. Source: Internal visualization tool.

The formal definition of the stage sets uses the position of nodes relative to production. Let $\tau : V_C \to \{\text{Proc}, \text{Prod}, \text{Cust}\}$ denote the node type (Procurement, Production, Customer transport/product), and let $P = \{v \in V_C : \tau(v) = \text{Prod}\}$ be the set of production nodes. Using reachability analysis on G_C, where $v \xrightarrow{*} p$ denotes that v can reach p via a directed path, define:

- $U_P = \left\{v \in V_C : \exists p \in P, v \xrightarrow{*} p\right\}$: nodes that are upstream of production (i.e., nodes from which at least one production node is reachable along the directed path)
- $D_P = \left\{v \in V_C : \exists p \in P, p \xrightarrow{*} v\right\}$: nodes that are downstream of production (i.e., nodes that are reachable from at least one production node along the directed path)

Note that every node can reach itself, meaning $p \xrightarrow{*} p$ holds for any node p. Consequently, all production nodes belong to both U_P and D_P.

Nodes in $U_P \setminus D_P$ are upstream of all production (stages 1-2), nodes in $D_P \setminus U_P$ are downstream of all production (stages 4-5), and nodes in $U_P \cap D_P$ lie between production stages (stage 3). Within these groups, the presence of direct procurement or customer edges determines the final stage assignment:

$$
\begin{aligned}
S_1 &= \{v \in U_P \setminus D_P : \tau(v) = \text{Proc} \vee \exists (u,v) \in E_C, \tau(u) = \text{Proc}\} \\
S_2 &= \{v \in U_P \setminus D_P : \tau(v) \neq \text{Proc} \wedge \neg\exists (u,v) \in E_C, \tau(u) = \text{Proc}\} \\
S_3 &= U_P \cap D_P \\
S_4 &= \{v \in D_P \setminus U_P : \tau(v) \neq \text{Cust} \wedge \neg\exists (v,w) \in E_C, \tau(w) = \text{Cust}\} \\
S_5 &= \{v \in D_P \setminus U_P : \tau(v) = \text{Cust} \vee \exists (v,w) \in E_C, \tau(w) = \text{Cust}\}
\end{aligned}
\tag{9.}
$$

Let $R = V_C \setminus (U_P \cup D_P)$ be the set of nodes without any production in their connected component. Let $R_1 = \{v \in R : \exists (u,v) \in E_C, \tau(u) = \text{Proc}\}$ be the subset with direct procurement input and $R_5 = \{v \in R : \exists (v,w) \in E_C, \tau(w) = \text{Cust}\}$ the subset with direct customer output. Then:

$$
\begin{aligned}
S_1 &\coloneqq S_1 \cup R_1 \\
S_5 &\coloneqq S_5 \cup (R_5 \setminus R_1) \\
S_4 &\coloneqq S_4 \cup (R \setminus R_1 \setminus R_5)
\end{aligned}
\tag{10.}
$$

Following these definitions every node is assigned correctly to their stage in the supply chain as described in Table 2. These sets S_1 to S_5 are pairwise disjoint and partition G_C, which is proven in Appendix 1. This property is essential for correctness, since it guarantees that each node is assigned to exactly one stage, preventing variables and constraints from being duplicated or lost during creation of the subgraphs and then subproblems from the stages. Resource nodes are excluded from the stage assignment and partitioning process beforehand, as their capacities may be shared across multiple subnetworks and require coordinated handling across partition boundaries. Their treatment is detailed in Section 5.5.

Subgraph Construction

A stage grouping $\mathcal{S} = \{S_1, S_2, ..., S_k\}$ partitions the five stages into k groups, where each group becomes one subnetwork. The grouping is a configuration parameter that determines the granularity of decomposition. For example:

- $\mathcal{S} = \{\{1,2,3,4,5\}\}$: No decomposition (monolithic)
- $\mathcal{S} = \{\{1,2\},\{3,4,5\}\}$: Two subnetworks (upstream vs. downstream)
- $\mathcal{S} = \{\{1,2\},\{3\},\{4,5\}\}$: Three subnetworks (sourcing, production, distribution)

Given the stage function $s : V \to \{1, ..., 5\}$, mapping a node to its stage and the grouping $\mathcal{S}$, the node assignment function $\sigma : V \to \{1, ..., k\}$ maps each node to its subnetwork:

$$
\sigma(v) = i \quad \text{where} \quad s(v) \in S_i \tag{11.}
$$

The subgraph of the subnetwork i, $G_i = (V_i, E_i)$ contains:

$$V_i = \{v \in V : \sigma(v) = i\} \tag{12.}$$

$$E_i = \{(u, v) \in E : u \in V_i \wedge v \in V_i\} \tag{13.}$$

Every edge $(u, v) \in E$ falls into exactly one of two categories: either both endpoints belong to the same subnetwork ($\sigma(u) = \sigma(v)$), in which case the edge is included in E_i, or the endpoints belong to different subnetworks ($\sigma(u) \neq \sigma(v)$), making it a cut edge. Cut edges require special handling through cut harmonization, but no edge is lost in the decomposition.

Cut Harmonization

Cut edges $(u, v) \in E$ where $\sigma(u) \neq \sigma(v)$ represent material flow between subnetworks. These edges cannot simply be removed, because they are the coupling between the subnetworks. To enable the exchange of materials across partition boundaries, the framework introduces "link nodes". Link nodes are designated location-product nodes that serve as the interface between adjacent subnetworks.

By the structure of supply chain graphs (see Section 2.2.1), every edge in the material flow connects a source-of-supply node (Production, Transport, Procurement) to a location-product node, or vice versa. Customer nodes form a special case but a customer transport node and a customer product node are always assigned to the same stage, so they cannot appear on cut edges. Consequently, every cut edge has at least one location-product node as an endpoint. These location-product nodes on cut edges form the set of link nodes $\mathcal{L}$:

$$\mathcal{L} = \{\ell \in V : \ell \text{ is a location-product node} \wedge \exists (u, v) \in E, \sigma(u) \neq \sigma(v), \ell \in \{u, v\}\} \tag{14.}$$

Each link node $\ell \in \mathcal{L}$ is replicated in both adjacent subnetworks. Consider an edge from production node $p \in G_i$ to link node $\ell \in G_j$. The node ℓ is added to G_i and an edge from p to ℓ is added in G_i. The link node ℓ now serves as the interface coupling between the subnetworks.

For each link node $\ell \in \mathcal{L}$ in the downstream subnetwork G_j, a virtual external procurement node v_ℓ^{virt} is created and connected to ℓ. This virtual node represents the supply that will flow from the upstream subnetwork to the downstream subnetwork. Therefore when solving the downstream subnetwork individually this virtual procurement node v_ℓ^{virt} gets treated as a supply point and the amount of the requested supply of the bottom subnetwork can be propagated to the upper subnetwork (see Section 5.3).

Constraint Deduplication and Feasibility

The replication of link nodes introduces the challenge that the same location-product now appears in two subnetworks, which would lead to duplicated constraints and variables and therefore capacities if not handled correctly. In the original monolithic formulation, each location-product has exactly one inventory balance constraint and one inventory variable. After replication, just generating constraints for both copies would double-count inventory capacity, initial stock, and storage costs.

To preserve feasibility, the framework assigns all inventory-related constraints exclusively to the upstream copy of each link node $\ell \in \mathcal{L}$ (in subnetwork G_i). The downstream copy (in subnetwork G_j) is marked as "non-storable", which leads to no constraint for inventory and no storage variable. The downstream copy thus functions as a pure flow-through point. This design ensures that the combined solution across subnetworks will maintain feasibility with respect to the original formulation.

5.3 Two-Phase Solve Approach

The coordination of subproblems across the decomposed network requires a systematic approach that respects the dependencies between subnetworks while ensuring solution feasibility. In contrast to iterative decomposition methods that require multiple passes to achieve convergence, the proposed approach consists of exactly two traversals: a bottom-up phase that propagates demand information from distribution toward sourcing, followed by a top-down phase that allocates actual supplies from sourcing toward distribution.

The bottom-up phase solves each subnetwork assuming unlimited supply at its virtual procurement cut points, to determine the demand requirements that should be satisfied by the upstream subnetwork. Subsequently, the top-down phase resolves each subnetwork with the actual supply capacities determined by its upstream subnetworks, yielding a feasible allocation solution throughout the network. This two-pass approach eliminates the convergence uncertainty of iterative approaches while guaranteeing feasibility through the hierarchical propagation of constraints in form of demands and actual supplies.

Meta Graph Construction

While handeling cut-edges, we construct a meta graph $M = (\mathcal{V}_M, \mathcal{E}_M)$, that captures the dependencies between the k subnetworks:

$$\mathcal{V}_M = \{1, 2, ..., k\} \tag{15.}$$

$$\mathcal{E}_M = \{(i, j) : \exists (u, v) \in C \text{ with } \sigma(u) = i, \sigma(v) = j\} \tag{16.}$$

The meta graph is a multi-graph allowing multiple edges between the same pair of subnetworks, one per distinct link node. The meta graph defines the solving order for both phases. A topological sort of M yields a sequence in which each subnetwork appears only after all of its successors, establishing the correct execution order for the solve of the corresponding subproblems in the bottom-up phase. Conversely, the top-down phase requires all upstream supplies to be determined before a subnetwork can be solved. The reversed topological order therefore provides the correct execution sequence for this case. Figure 4 shows an example meta graph which would result in $(2, 1, 0)$ for the bottom-up solve and $(0, 1, 2)$ for the top-down solve.

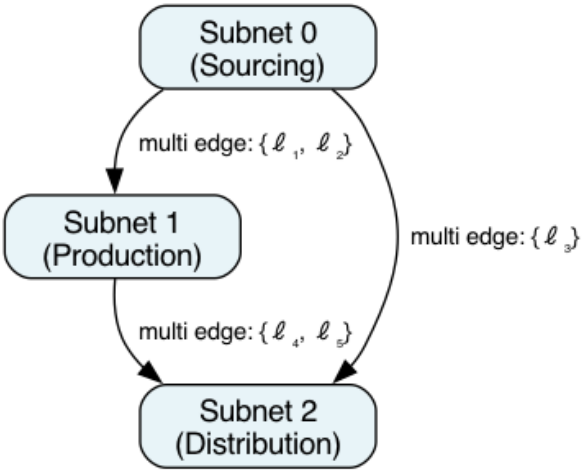


Figure 4: Example meta graph M for three subnetworks

Demand Propagation (Bottom-Up Phase)

The bottom-up phase determines how much material each subnetwork requires from its upstream neighbors. Subnetworks are solved in reverse topological order, starting from distribution (closest to customers) and proceeding toward sourcing. Each subnetwork G_i is solved as an independent scemario with virtual procurement nodes representing the possible, currently infinte supply from upstream subnetworks. This means the modified constraint for link node ℓ in the subnetwork G_i becomes:

$$\mathrm{VS}_\ell(t) = \mathrm{VS}_\ell(t-1) + \mathrm{VP}_\ell(t) + ..., \tag{17.}$$

with decision variable $\mathrm{VP}_\ell(t)$ that represents the procurement from the upper subnetwork, allowing the solver to "spawn" material at this location, that is propagated to the supplying subnetwork in the following way. Let $\mathrm{VP}_v^{\mathrm{virt}}(t)$ denote the planned procurement quantity at virtual node v by subnetwork G_i. This quantity represents the demand that subnetwork G_i requests from its upstream neighbor. After solving G_i, the demand is propagated to the upstream subnetwork G_j by creating a location demand at the corresponding boundary location-product node ℓ:

$$\mathrm{DQ}_{\ell,t} = \mathrm{VP}_{v_\ell^{\mathrm{virt}}}(t) \tag{18.}$$

where $\mathrm{DQ}_{\ell,t}$ is the demand at location-product ℓ in period t in subnetwork G_j, and $\mathrm{VP}_{v_\ell^{\mathrm{virt}}}(t)$ is the external procurement quantity from the virtual procurement node connected to ℓ in G_i. This demand is added to the upstream subnetwork as a demand target with constraint $\mathrm{VP}_\ell(t) \leq \mathrm{DQ}_{\ell,t}$. Then, the solve of subproblem G_j determines the optimal delivered quantity $\mathrm{VL}_\ell(t)$, which appears both in the inventory balance as an outflow and in the objective function with a penalty term $\mathrm{CD}_\ell \cdot \big(\mathrm{DQ}_{\ell,t} - \mathrm{VL}_\ell(t)\big)$ for any unfulfilled portion. The non-delivery penalty CD_ℓ is derived from an aggregated non-delivery cost, described in the following Section 5.4.4.

Since the virtual procurement nodes have unlimited capacity during the bottom-up phase, the solver determines ideal demand requirements without supply constraints, propagating demands backward through the supply chain from customer demands to procurements. This may result in demands that exceed what upstream subnetworks can actually provide. When an upstream subnetwork cannot fully satisfy a downstream demand, the downstream subnetwork's solution becomes temporarily infeasible. The top-down phase corrects this by re-solving each subnetwork with the actual supplies delivered by its upstream neighbors, ensuring global feasibility.

Supply Propagation (Top-Down Phase)

The top-down phase then allocates actual supplies based on the supply decisions of upstream subnetworks. Subnetworks are solved in topological order, starting from sourcing and proceeding toward distribution. The first subnetwork retains its bottom-up solution unchanged, as it has no upstream dependencies which could need correction.

For each subsequent subnetwork G_i, the virtual procurement nodes are replaced with fixed supply quantities derived from the upstream solutions. $\mathrm{VL}_\ell^{G_j}(t)$ denotes the

quantity actually delivered to location-product ℓ in period t in the upstream subnetwork G_j. The top-down phase injects this quantity as a constant monome into the inventory balance constraint (Equation 6). The modified constraint for link node ℓ in the downstream Subnetwork G_i becomes:

$$\mathrm{VS}_\ell(t) = \mathrm{VS}_\ell(t-1) + \mathrm{VL}_\ell^{G_j}(t) + ... \tag{19.}$$

where $\mathrm{VL}_\ell(t)$ is now a fixed constant rather than a decision variable. This fixed supply term represents the actual material availability from the upstream subnetwork. The virtual external procurement rule for the boundary node is removed. The top-down solve may produce a different, worse solution compared to the bottom-up phase, since the actual supply may be less than the requested demand.

5.4 Link Node Heuristics

The subnetworks have no information about cost-structure or non delivery penalties of neighboaring subnetworks. While the stated design already produces valid solutions for any problem, the solutions can be improved by providing more information to the different subproblems. Therefore the virtual procurement nodes introduced at cut edges must be parameterized without access to upstream subproblem solutions. Each virtual procurement gets assigned four parameters:

- maximum flow capacity over this cut-edge,
- leadtime aggregation as the earliest availability period,
- virtual marginal procurement cost,
- virtual non-delivery penalty.

Computing exact values would necessitate solving the upstream subproblems first, defeating the purpose of decoupled optimization. The framework therefore employs conservative heuristic estimates that keep the solution space of the combined subnetwork solution the same, but restrict the solution spaces of the subproblems in the bottom-up phase, to get better virtual demand estimates. These parameters are assigned to the virtual procurement nodes created for each link node and will then be transformed into constraints during the *MILP* formulation. The following sections describe each heuristic and how the computed values enter the subproblem formulation.

The aggregation heuristics must respect the structural semantics of different node types. Let $\mathrm{pred}(v) = \{u \in V : (u, v) \in E\}$ denote the set of immediate predecessors of node v, and let $\mathrm{succ}(v) = \{w \in V : (v, w) \in E\}$ denote the set of immediate successors. Two

distinct behaviors are distinguished based on the logical relationship between a node and its predecessors:

- **OR-nodes**: Location-product and customer-product nodes represent points where material can arrive from multiple alternative sources. One source would be enough to fulfill demand.
- **AND-nodes**: Production nodes require inputs from all predecessors simultaneously, consuming components according to the *BOM* coefficients $\mathrm{PR}_{o,s,p}$ defined by the production model, where negative values indicate consumed materials and positive values indicate produced materials.

5.4.1 Production Cost Aggregation

The virtual procurement nodes require cost parameters representing the expense of obtaining materials from upstream subnetworks. Without these parameters, the downstream optimization would treat upstream supply as cost-free, leading to over-procurement. Production cost aggregation computes an estimated procurement cost for each boundary product based on the accumulated costs along the upstream supply paths.

Let c_v denote the marginal cost of node v, representing the direct operational cost per unit on the Operation (e.g. Transportcost, Productioncost, Procurementcost). The aggregation proceeds in topological order from sources to sinks. The aggregated cost c_v^{agg} is computed as:

$$c_v^{\mathrm{agg}} = \begin{cases} c_v & \text{if } \mathrm{pred}(v) = \emptyset \\ c_v + \min_{u \in \mathrm{pred}(v)} c_u^{\mathrm{agg}} & \text{if } v \text{ is an OR-node} \\ c_v + \sum_{u \in \mathrm{pred}(v)} |\mathrm{PR}_{(u,v)}| \cdot c_u^{\mathrm{agg}} & \text{if } v \text{ is an AND-node} \end{cases} \tag{20.}$$

For OR-nodes, the minimum reflects the optimizer's ability to choose the cheapest supplier. For AND-nodes, the sum scaled by *BOM* coefficients captures the total input cost. The aggregated cost constitutes an underestimate of the true procurement cost, as the minimum operator assumes the cheapest path is always available. Capacity constraints may force later selection of more expensive alternatives in practice, but for the purpose of guiding the optimization toward cheaper decisions, this approximation is sufficient. The computed c_v^{agg} is assigned as the procurement cost parameter of the virtual procurement node, entering the objective function as a per-unit cost term.

5.4.2 Lead Time Aggregation

Lead time l_v represents the number of periods required from the initiation of an operation at node v until the resulting material becomes available. When a subnetwork boundary is introduced, the downstream subnetwork loses visibility into upstream timing needs. The virtual procurement node must therefore carry an aggregate lead time reflecting the minimum time required for material to traverse the upstream network upon the point of the link node.

The aggregation proceeds in topological order. The aggregated lead time l_v^{agg} is computed as:

$$l_v^{\text{agg}} = \begin{cases} l_v & \text{if } \text{pred}(v) = \emptyset \\ \min_{u \in \text{ pred}(v)} l_u^{\text{agg}} + l_v & \text{if } v \text{ is an OR-node} \\ \max_{u \in \text{ pred}(v)} l_u^{\text{agg}} + l_v & \text{if } v \text{ is an AND-node} \end{cases} \tag{21.}$$

For OR-nodes, the minimum selects the fastest alternative sourcing path. For AND-nodes, the maximum identifies the critical path, since all inputs must be present before production can commence.

The computed l_v^{agg} at boundary nodes is assigned to virtual procurement nodes as their validity period. This aggregation provides an optimistic underestimate, since capacity constraints may still limit the flow in valid periods because of missing input materials.

5.4.3 Max Flow Aggregation

The max flow aggregation computes an upper bound on the maximum quantity that can flow through each cut edge per period. Without capacity bounds, the downstream optimization may request quantities exceeding what the upstream network can provide, which would then get restricted a lot by the top-down phase. The heuristic computes an upper bound on achievable throughput at each boundary node, conceptually related to the max-flow/min-cut theorem [38].

The aggregation proceeds in topological order from source nodes toward boundary nodes. Let f_v^{agg} denote the aggregated max flow at node v, and let f_v^{own} denote the node's own capacity constraint (e.g. Maximum Transport, Maximum Production, Resource Capacity). The aggregated max flow is computed as:

$$f_v^{\text{agg}} = \begin{cases} f_v^{\text{own}} & \text{if } \text{pred}(v) = \emptyset \\ \sum_{u \in \text{ pred}(v)} f_u^{\text{agg}} & \text{if } v \text{ is an OR-node} \\ \min_{u \in \text{ pred}(v)} \frac{f_u^{\text{agg}}}{|\text{PR}_{(u,v)}|} & \text{if } v \text{ is an AND-node} \end{cases} \quad 22.$$

For OR-nodes, the sum reflects that any combination of sources can contribute to total flow. For AND-nodes, the minimum identifies the bottleneck input, scaled by the *BOM* coefficient to convert input capacity to output capacity. The final aggregated value incorporates any local resource constraints: $f_v^{\text{agg}} = \min(f_v^{\text{agg}}, f_v^{\text{own}})$.

The aggregated max flow is deliberately an overestimate. The heuristic does not account for material being consumed by other downstream paths or shared resource contention, effectively assuming all upstream capacity is available for the specific path being computed. This overestimation ensures that feasible solutions are not prematurely excluded and the top-down phase subsequently corrects the allocation based on actual upstream supplies. The computed f_v^{agg} enters the subproblem as an upper bound constraint on the virtual procurement variable.

5.4.4 Non-Delivery Cost Aggregation

Non-delivery cost represents the penalty incurred when customer demand cannot be fulfilled. Virtual procurement nodes in upstream subnetworks have no visibility into the downstream demand structure they serve. Without propagating penalty information upstream, virtual link nodes cannot distinguish between demands of varying importance. The non-delivery cost aggregation propagates penalty costs backward from demand nodes to their upstream suppliers.

Unlike the other heuristics, this aggregation operates in reverse topological order, from sink nodes toward source nodes. Let p_v denote the non-delivery penalty at a demand node v. The aggregated non-delivery cost p_v^{agg} is computed as:

$$p_v^{\text{agg}} = \begin{cases} p_v & \text{if } \text{succ}(v) = \emptyset \\ p_v + \frac{1}{|\text{succ}(v)|} \sum_{w \in \text{ succ}(v)} \frac{p_w^{\text{agg}}}{|\text{PR}_{v,w}|} & \text{otherwise} \end{cases} \quad 23.$$

The inverse *BOM* scaling accounts for material conversion ratios, since, when multiple units of a component produce one unit of output, failing to deliver one component unit affects on average only a fraction of the final product. The average over successors distributes the downstream penalty proportionally when a node feeds multiple paths.

This heuristic is inherently imprecise since it remains unknown which specific customer demand will ultimately be fulfilled by delivering a particular intermediate product. Furthermore, failing to deliver one component of a production process prevents the entire output from being produced, not merely the fraction implied by the *BOM* coefficient. Nevertheless, the aggregated penalty provides the solver with directional guidance to prioritize fulfilling demands with higher downstream criticality. The computed p_v^{agg} is assigned as the non-delivery penalty CD_ℓ of the virtual demand node created at the link node, entering the objective function as a penalty term for unfulfilled virtual demand.

5.5 Inter Subnet Constraint Handling

The graph partitioning described in the previous sections creates subnetworks that can be solved independently, but certain constraints in the original formulation cannot be assigned to a single subproblem. As introduced in Section 2.2.2, resource nodes impose shared capacity limits across multiple operations through constraints of the form $\sum_{o \in O_{s_r}} \text{RR}_{o,r} \cdot \text{VO}_{o,s}(t) \leq \text{RC}_r(t)$, where O_{s_r} is the set of production models using the resource's capacity. When operations from different subnetworks share the same resource, this constraint spans multiple subproblems and cannot be enforced locally. Without coordination, each subnetwork would assume access to the full resource capacity, leading to collective over-allocation that violates the original constraint. This section presents three approaches for allocating shared resource capacity across subnetworks. The choice of allocation strategy is a configuration parameter of the framework and its impact on solution quality and computational performance is evaluated empirically in the experiments.

5.5.1 Equal Split

The equal split strategy provides a straightforward mechanism for distributing shared resource capacity across subnetworks. The capacity is divided equally among all sharing subnetworks, regardless of their actual requirements. This approach requires only structural information about which subnetworks reference each resource, without knowledge of consumption patterns or optimization outcomes.

Let r denote a shared resource with capacity $\text{RC}_r(t)$ in period t, and let $N_r = \{i_1, i_2, ..., i_m\}$ be the set of subnetwork indices that contain operations consuming r. The capacity allocated to each subnetwork $i \in N_r$ is:

$$\mathrm{RC}_r^{(i)}(t) = \frac{\mathrm{RC}_r(t)}{|N_r|} \tag{24.}$$

The equal split approach offers simplicity and guaranteed feasibility since each subnetwork receives a fraction of the original capacity and these fractions sum to one, the combined usage across all subnetworks cannot exceed the original constraint. However, this strategy can result in inefficient utilization when subnetworks have heterogeneous demands.

5.5.2 First-Come First-Serve

The first-come first-serve (FCFS) strategy allocates resource capacity dynamically based on actual usage during the top-down phase. In the bottom-up phase, all subnetworks receive access to the full original capacity. In the top-down phase, the upstream subnetwork is solved first with full capacity access, and its actual consumption is recorded. Each subsequent downstream subnetwork then receives only the remaining capacity:

$$\mathrm{RC}_r^{(i)}(t) = \mathrm{RC}_r(t) - \sum_{j \text{ solved before } i} \sum_{o \in O_j} \mathrm{RR}_{o,r} \cdot \mathrm{VO}_o^{(j)}(t) \tag{25.}$$

This approach aligns capacity allocation with material flow direction. However, upstream subnetworks may consume all available capacity, leaving downstream subnetworks with insufficient capacity to fulfill demands.

5.5.3 LP-Relaxation

The LP-relaxation strategy provides an informed capacity allocation by first solving the linear programming relaxation of the entire monolithic problem. As established in Equation 2, the LP relaxation removes all integrality constraints, allowing the problem to be solved efficiently in polynomial time [6]. The resulting fractional solution provides guidance on how capacity would be utilized under optimal continuous conditions.

The allocation proceeds in two steps. First, the LP relaxation of the complete supply chain problem is solved, yielding optimal fractional values for all decision variables. For each shared resource, the LP solution reveals how much capacity each operation consumes.

Second, the capacity is distributed across subnetworks proportionally based on the LP-derived usage. For a shared resource r with capacity $\mathrm{RC}_r(t)$, let N_r be the set of subnetwork indices containing operations that consume r. The total LP usage by

operations in subnetwork i is $U_i^{\text{LP}} = \sum_{o \in O_i} \text{RR}_{o,r} \cdot \text{VO}_o^{\text{LP}}(t)$. The capacity allocated to each subnetwork $i \in N_r$ is:

$$\text{RC}_r^{(i)}(t) = \text{RC}_r(t) \cdot \frac{U_i^{\text{LP}}}{\sum_{j \in N_r} U_j^{\text{LP}}} \tag{26.}$$

Equation 26 distributes the entire original capacity $\text{RC}_r(t)$ across all subnetworks that have operations consuming the resource, using the LP solution only to determine the allocation ratios. That means that even if the LP solution does not use the whole available capacity, each subnetwork receives its proportional share of the full capacity. Since all proportianal shares sum to 1, this ensures that unused capacity in the LP solution remains available for the integer subproblems, even if it was not used in the LP-Solution.

This approach offers several advantages over the alternatives. Unlike equal split, the allocation reflects actual optimization priorities rather than treating all subnetworks uniformly. Unlike FCFS, all subnetworks receive their allocation simultaneously based on global information, avoiding the sequential bias where solve order determines capacity access. The primary limitation is that resource consumption in the LP-solution may differ a lot from the actual problem solution with integer constraints and that solving the LP-Relaxation can also take a long time.

5.6 Solution Combination

After the top-down phase completes, each subnetwork has produced a local solution. These solutions must be combined into a single solution for the original monolithic problem. The combination process addresses two categories of variables:

- unique variables that exist in exactly one subnetwork,
- link node variables at cut edges.

Unique Variables

The majority of decision variables in the original formulation appear in exactly one subnetwork. Decision variables for nodes that are not link nodes are directly adopted from their respective subnetwork solutions. These variables require no special handling since their constraints are entirely contained within a single subnetwork.

Link Nodes

As described in Section 5.2, link nodes $\ell \in \mathcal{L}$ are replicated across adjacent subnetworks during cut harmonization. The upstream copy in subnetwork G_i retains all inventory-related constraints and variables, while the downstream copy in G_j functions as a flow-through point with no inventory constraints.

During the two-phase solve, virtual demands and virtual procurements are introduced at link nodes to coordinate the subnetworks. In the combined solution, these virtual elements are removed. The upstream copy's solution provides all inventory and constraint-related values for the link node. The delivered quantity $\mathrm{VL}_\ell(t)$ from the upstream subnetwork becomes the actual inventory outflow to the downstream subnetwork, satisfying the original material inflow constraint without requiring an explicit demand variable in the combined solution:

$$\mathrm{VS}_{s,p}^{\text{combined}}(t) = \mathrm{VS}_{s,p}^{(i)}(t) \quad \text{for } \ell \in \mathcal{L} \tag{27.}$$

This works because the decomposition assigns all inventory constraints exclusively to the upstream copy. The downstream copy has no inventory variable and constraint that would conflict, ensuring the combined solution satisfies the original inventory balance.

The resource capacity splitting strategies ensure that each subnetwork respects its allocated capacity share. Since the allocated shares sum to at most the original capacity, the combined resource consumption cannot exceed the original resource constraint $\mathrm{RC}_r(t)$.

The combination step is straightforward because the top-down phase has already resolved all cross-boundary dependencies since each subnetwork was solved with the actual supplies provided by its upstream neighbors, ensuring that the combined solution is feasible with respect to the original formulation.

5.7 Runtime Allocation

When solving large-scale supply network instances, a total runtime budget is specified to limit computational cost. In the monolithic case, this budget is allocated to a single *MILP* solver invocation. Under decomposition, the total budget must be distributed across multiple subproblem solves. When dividing into k subnetworks the bottom-up phase needs k *MILP* solve runs and the top-down phase needs $k-1$ *MILP* solve runs, because the first subnetwork retains its solution unchanged. This leads to $2k-1$ solve

operations in total. A time distribution strategy needs to address that problems with less variables will find better solutions faster and might be solved to optimality before the allocated time expires. Bigger problems on the other hand might need more time to find even one possible solution, which is crucial to continue the whole solve, since when one problem does not find a solution, the whole run cannot be continued.

A complexity-weighted allocation strategy is therefore introduced, that distributes runtime proportionally to estimated subproblem difficulty. Since the exact model statistics are only available once the mathematical model is built, estimates depending on the graph nodes and their master data are used. Let n_i^{cont} denote the estimated number of continuous variables and n_i^{disc} the estimated number of discrete variables in subnetwork i. The complexity of solving a *MILP* is dominated by the combinatorial search over discrete variables, while continuous variables contribute linearly through the LP relaxations solved at each branch-and-bound node. The estimated complexity score can be defined as:

$$C_i = \left(n_i^{\text{cont}} + n_i^{\text{disc}}\right) + 5 \cdot \left(n_i^{\text{disc}}\right)^2, \tag{28.}$$

so continuous variables contribute linearly to the complexity, while discrete variables have quadratic scaling with a higher coefficient. These scalings reflect the exponential growth of branch-and-bound tree size with discrete variables and the polynomial cost of LP relaxations. This formula is used and validated in other SAP decomposition modules [12]. The estimation formulas for counting discrete and continuous variables per node type are detailed in Appendix 2.

The complexity scores are computed once before solving begins and normalized to obtain time shares $s_i = \frac{C_i}{\sum_{j=1}^{k} C_j}$ for each subnetwork. However, the actual runtime allocation is performed dynamically during the solving process based on remaining subproblems and previously consumed time.

Let T_{rem} denote the remaining runtime budget and S_{rem} the sum of time shares for all unsolved subnetworks. When subnetwork i is about to be solved, its allocated runtime is:

$$t_i = T_{\text{rem}} \cdot \frac{s_i}{S_{\text{rem}}} \tag{29.}$$

After solving subnetwork i, let t_i^{used} denote its actual solver runtime. The remaining budget is updated as $T_{\text{rem}} := T_{\text{rem}} - t_i^{\text{used}}$, and subnetwork i's time share is removed from the remaining share pool: $S_{\text{rem}} := S_{\text{rem}} - s_i$. This dynamic reallocation ensures that any

time saved by a subnetwork that solves faster than expected is redistributed proportionally among the remaining unsolved subnetworks according to their complexity shares.

To prevent excessively small allocations to low-complexity subnetworks, a minimum share threshold $s_{\min} = 0.1$ (10%) is enforced, ensuring each subnetwork receives at least 10% of the total budget regardless of its complexity. If a subnetwork's complexity-based share s_i falls below $s_{\min}$, it is raised to $s_{\min}$. The required additional share is taken proportionally from subnetworks whose shares exceed $s_{\min}$. Specifically, let $D = \sum_{s_i < s_{\min}} (s_{\min} - s_i)$ be the total deficit and $S = \sum_{s_i \geq s_{\min}} (s_i - s_{\min})$ be the total surplus available. Each subnetwork j with $s_j \geq s_{\min}$ has its share reduced by:

$$s_j := s_j - D \cdot \frac{s_j - s_{\min}}{S} \tag{30.}$$

This redistribution preserves the relative ordering of shares above the minimum while guaranteeing all subnetworks receive sufficient time. The normalized shares still sum to 1.0 and are used throughout both solving phases.

The complexity-weighted approach offers two advantages. First, it allows harder subproblems to utilize more computational time, potentially improving solution quality where it matters most. Second, it prevents easy subproblems from consuming time they do not need.

However, complexity-based allocation has limitations. The complexity score is a heuristic estimate that may not accurately predict actual solver runtime, which depends on a lot more things than only variable counts. Furthermore, if the complexity estimate underallocates time to a critical subproblem, that subproblem may terminate prematurely with a poor or no solution, degrading the quality of the overall decomposed solution or making it impossible to solve.

6 Implementation

This chapter describes the technical implementation of the decomposition framework. The system is realized as an external orchestration layer written in Python that operates independently of the SAP IBP Supply Planning optimization engine while leveraging its solver infrastructure through an HTTP service interface.

6.1 System Architecture

The decomposition framework is implemented as an external orchestration system that coordinates multiple solver invocations without modifying the underlying optimization engine. This architectural decision stems from the requirement to preserve the correctness and modularity of the production solver while enabling experimental decomposition strategies. Embedding decomposition logic directly into the optimization engine would require extensive modifications to a complex codebase and would complicate the process for the *Proof-of-Concept (PoC)* of this thesis.

Figure 5 loosely illustrates the overall system architecture, showing the interaction of local modules and the remote optimization service.

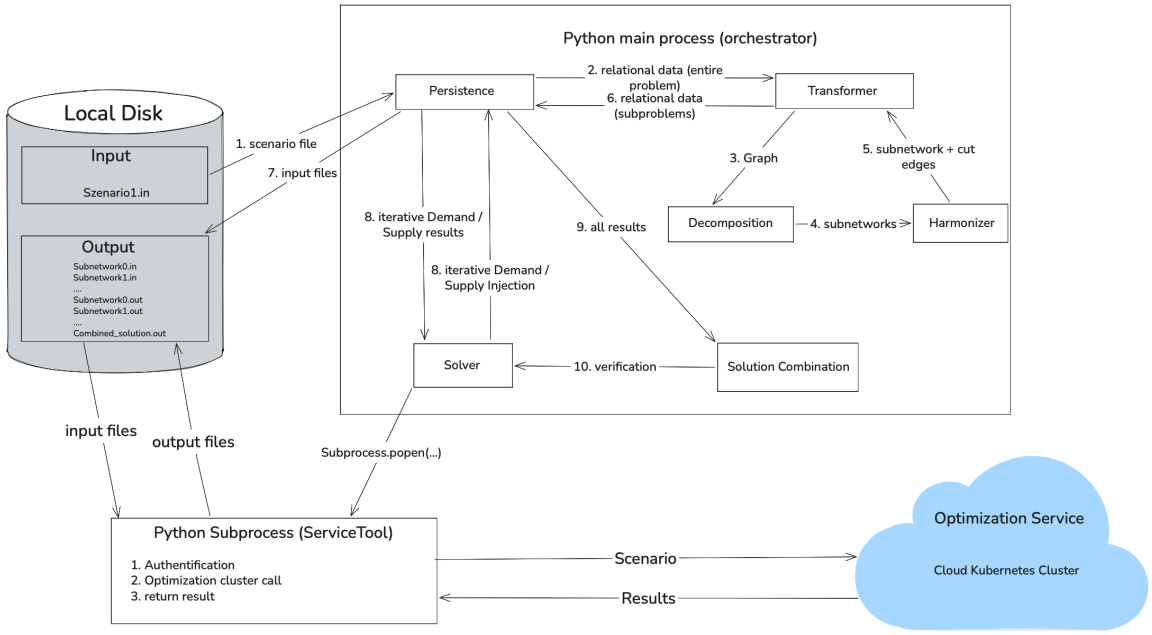


Figure 5: System architecture of the decomposition framework showing the orchestration layer and solver service interaction

Module Overview

The framework is organized into the following packages:

- **domain**: Contains domain models, data mappings, and configuration for supply chain entities. Defines node type enumerations, node and edge resolution from database tables, and heuristic configuration parameters as described in Section 5.4 and Section 5.5.
- **algorithms**: Implements the core graph algorithms. The `transformer` module handles bidirectional conversion between database tables and graph representation as described in Section 5.1. The `decomposition` subpackage contains `static_decomposer` for stage classification and graph partitioning (Section 5.2) and `subnetwork_harmonizer` for cut edge handling and meta-graph construction. The `solver` subpackage implements `static_solver` for the two-phase solving approach and `executor` for invoking the Optimization Service (Section 5.3), `solution_harmonizer` for combining subnetwork solutions (Section 5.6), and `constraint_propagation` for cross-boundary constraint handling. The `initial_heuristics` subpackage provides `aggregation_propagation` for computing link node parameters (Section 5.4).
- **persistence**: Handles file I/O. Includes `file_parser` and `file_writer` for database serialization.
- **orchestration**: Coordinates the overall workflow through the `orchestrator` module, which sequences the transformation, decomposition, solving, and solution combination phases as described in Section 5.3.

ServiceTool Integration

The framework communicates with the SAP *IBP* optimization service through a command-line interface tool called ServiceTool. This tool acts as a bridge between the local Python environment and the cloud-based optimization infrastructure. ServiceTool performs OAuth2 authentication against the service cluster, then uploads the input file along with the solver configuration. After the optimization completes on the remote infrastructure, ServiceTool downloads the solution archive containing the result tables. ServiceTool is invoked as a subprocess from Python using the `subprocess.Popen` interface with streaming output capture.

Error Handling and Recovery

The executor module implements retry logic with exponential backoff for transient failures such as network timeouts or temporary service unavailability. Permanent failures, such as solver time limit reached or infeasible problems, are detected through exit code analysis in the solver output and raise dedicated exceptions that terminate the current solve attempt without retry.

The framework architecture supports recovery functionality through checkpointing of intermediate results. Each subnetwork solution is persisted to disk immediately after completion, which enables recovery from failure after every fully finished optimization run. The system can detect which subnetwork was the last one completely solved and continue from there.

6.2 Data Handling and Technologies

The framework operates on data stored as files on the local filesystem. This file-based approach enables inspection of intermediate subnetwork solutions and provides natural checkpoints for the solving process.

Python was selected as the implementation language for several reasons [39]. The ecosystem provides libraries for graph algorithms and large data operations. The language's flexibility supports rapid prototyping and fast development of decomposition approaches. Furthermore, Python's subprocess management enables integration with the ServiceTool command-line interface.

The framework employs the following primary libraries:

- **NetworkX** [40]: Used for graph representation and algorithms. The supply chain network is modeled as a `DiGraph`. The library contains several graph algorithms like traversals or condensation which are used throught the framework.
- **Polars** [41]: Used for tabular data manipulation. Polars provides high-performance DataFrame operations for filtering and concatinating the master data tables and solver result tables. The library is used throughout SAP Supply Planning scripts and has direct integration with the table data format.
- **NumPy** [42]: Used for numerical computations on graph node and edge attributes, particularly for the aggregation heuristics.

After partitioning the supply chain graph into subgraphs as described in Section 5.2, each subnetwork must be converted into a standalone scenario file that the solver can process independently. The framework achieves this through a table filtering mechanism that extracts the data rows relevant to each subnetwork from the entire monolithic file. Each table in the data schema defines retain-if rules that specify which graph node types are referenced by a row in the table. The `columns` mapping specifies which table columns must match the corresponding node attributes for a row to be retained. Listing 1 shows an example of such a rule for the production source item table, which stores the bill of materials for production rules. This table references both the production node through location and source, and the component's location-product node. Multiple retain-if items in the list are AND-concatenated, meaning a row is only retained if it matches nodes from all specified types in the subgraph.

```
RetainIfList(
    items=[
        RetainIf(
            node_type=NodeType.PRODUCTION,
            columns={"LOCID": "LOCID", "SOURCEID": "SOURCEID"}
        ),
        RetainIf(
            node_type=NodeType.LOCATION_PRODUCT,
            columns={"COMPONENTPRDID": "PRDID", "LOCID": "LOCID"}
        ),
    ],
    mode=RetainIfMode.AND
)
```

Listing 1: Retain-if configuration for the production source item table

When generating the scenario file for subnetwork i, the framework collects all node identifiers and attributes present in subgraph G_i, then filters each table to retain only rows whose key columns reference nodes within that subgraph. The filtered tables are then serialized into a binary scenario file that represents a standalone optimization scenario.

This pipeline ensures that each subproblem receives exactly the data required to optimize its portion of the supply chain, with cut edges handled through the virtual procurement mechanism described in Section 5.2.

The underlying optimization engine uses Gurobi as its *MILP* solver. Gurobi is a commercial solver known for its performance on large-scale mixed-integer programs [10]. Gurobi is used for most SAP Supply Planning optimization runs due to the good performance. Therefore it will also be used here. The complete solver parameter configuration is documented in Appendix 3.

The optimization problems are solved on the SAP Performance Optimization Cluster, a cloud-based infrastructure. Although the actual hardware can differ based upon the assigned computational node, experiments showed that different runs lead to roughly the same performance values. All experiments in this thesis, both monolithic and decomposed solves, are executed on this same infrastructure to ensure fair comparison.

6.3 Solution Verification

The decomposition framework produces solutions through a sequence of coordinated subproblem solves. These solutions are verified against the original monolithic formulation to confirm that the decomposed solution is feasible for the original problem and to obtain the true objective function value under the complete cost structure.

The verification procedure takes the original monolithic problem and adds the values from the decomposed solution as pseudo hard constraints. Specifically, the production quantities VO, transport quantities VT, and procurement quantities VP from the decomposed solution are injected into the problem formulation.

Pseudo hard constraints work by adding a penalty term to the objective function for any deviation from the target value. For each decision variable VX_i with decomposed solution value VX_i^*, the solver adds:

$$M \cdot |\mathrm{VX}_i - \mathrm{VX}_i^*| \tag{31.}$$

to the objective function where M is a large penalty coefficient. This soft constraint approach allows the solver to deviate from the injected values if necessary to maintain feasibility, while strongly preferring the original solution values. In practice, the penalty is large enough that the solver converges to the adjusted values unless doing so would violate hard constraints, because the verification runs are solved to optimality. Therefore the decomposed solution is valid if no pseudo hard constraints are violated.

The verification solve completes rapidly compared to the original optimization. With the primary decision variables fixed, the remaining problem reduces to computing the

implied values of dependent variables and evaluating the objective function. The solver does not need to explore the combinatorial space of production and transport decisions, as these are predetermined by the injected constraints. In practice, no verification run has exceeded a runtime of 10 minutes. This efficiency enables verification to be performed after each decomposed solve to obtain the correct objective function value. The correct objective function value cannot be obtained directly from the decomposed solutions, because, during the two-phase solve, each subproblem optimizes a modified objective that includes virtual costs and penalties for link nodes. These virtual terms guide the decomposition but do not correspond to actual costs in the original formulation. The verification problem uses the original objective function without any virtual terms. The resulting objective value represents the true cost of the decomposed solution as it would be evaluated by the monolithic solver to allow comparison.

7 Evaluation

This chapter presents the experimental evaluation of the proposed decomposition approach. The evaluation systematically compares solutions obtained through decomposition against monolithic solving across the ten benchmark scenarios introduced in Section 4.2.2, using the metrics defined in Section 4.2.1. After the general approach comparison, also the different configuration parameters of the decomposition approach are systematically varied to analyze their impact on runtime and solution quality. The parameter dimensions are “Subnetwork Granularity”, “Constraint Handling Modes” and “Heuristics”. Empirical observations and correlations identified during the evaluation are labeled as *Findings* and numbered sequentially. Each finding is then analyzed and interpreted in Section 8.

7.1 Experimental Setup

Each scenario is associated with a runtime budget that reflects the time limit used by the customer in production. Table 3 shows the runtime budgets for all benchmark scenarios. All experimental runs, regardless of decomposition configuration, use the same runtime budget for their respective scenario.

For the baseline comparison, the decomposition approach uses a fixed configuration to compare the general decomposition approach with the monolithic results. Subsequent sections then systematically vary each configuration parameter to investigate potential improvements over this baseline. The baseline used configuration is:

- Stage groups: {{1,2},{3},{4,5}}: three subnetworks corresponding to the supply chain stages procurement, production and distribution,
- Inter Subnet Constraints: Equal split,
- Heuristics: all active.

Scenario	Runtime Budget (s)	Runtime Budget (h)
S1	31000	8.6
S2	31000	8.6
S3	21600	6.0
S11	21600	6.0
S13	10800	3.0
S14	10800	3.0
S15	10800	3.0
S17	43200	12.0
S18	30000	8.3
S19	30000	8.3

Table 3: Runtime budgets for benchmark scenarios

7.2 Scenario Comparison

The primary evaluation compares the decomposition approach against the monolithic baseline across all benchmark scenarios. For each scenario, both approaches are given identical runtime budgets and solver configurations as defined in Section 4.2.

7.3 Subnetwork Granularity

The number of subnetworks presents a fundamental trade-off because finer granularity reduces subnetwork complexity but increases coordination overhead and the number of solver invocations. This section investigates how varying the number of subnetworks affects runtime and solution quality. The other configurations are fixed at Equal Split and all heuristics active.

Although the total number of possible stage groupings is $2^4 = 16$, several configurations are excluded from evaluation. Splitting between stages 1 and 2 provides no benefit because stage 1 consists solely of procurement operations. Since the cut-edge handling introduces a virtual procurement operation to the downstream subnetwork, the complexity of would not be reduced. The number of solver invocations scales with the number of subnetworks due to the two-phase solving approach. A configuration with five individual subproblems is therefore impractical. After excluding these cases, seven non-monolithic stage grouping configurations remain for evaluation. Figure 7 shows…

7.4 Constraint Handling Modes

This section evaluates the three resource capacity splitting modes and their impact on solution quality. The other configurations are fixed at stage groups $S = \{\{1, 2\}, \{3\}, \{4, 5\}\}$ and all heuristics on.

7.5 Heuristics

The decomposition framework employs four heuristics for aggregating information at subnetwork boundaries. This section quantifies the contribution of each heuristic to solution quality. The other configurations are fixed at stage groups $S = \{\{1, 2\}, \{3\}, \{4, 5\}\}$ and equal split for inter subnetwork constraints.

7.6 Summary

8 Discussion

- Produktionsszenarios => ungleiche Probleme, hilft wenig + oberes Problem leer, unteres Problem kleines LP, man verliert viel globales Optimum für wenig Perforamnce
- Topdown_subnetwork1 fast immer das Bottleneck Problem: Größtes Problem, sieht man auch schon stark bei S19Lp-relaxiert, die Probleme haben gleichviele nonzeros aber die fest gelieferten mengen entarten das Polyeder wodurch die solve zeit sehr lange wird. Und war auch bei dem einen toInspectRuns so., eventuell nicht so gut sichtbar bei Milps, aber in der LP relaxation extrem gut sichtbar und jedes MILP wird ja dann über häufige LP Relaxierung im B&B gelöst und das sieht man dann an der geringen Anzahl an nodes im top-down1, dadurch ist das Milp schlechter gelöst und weniger demand fulfillment rate, Begründung: aufgrund der Entartung gewinnt Barrier anstatt Simplex, da es mit Degerenacy kein Problem hat, aber danach ist das Crossover sehr langsam, aufgrund der entartung. Deshalb Idee: zu Simplex zwingen, das würde theoretisch auch besser mit Warmstart funktionieren.
- Gleichverteilung wird wohl besser sein, also wenn nicht nur Produktion diskret ist, sondern auch die Transporte oder halt einfach allgemein die Probleme etwa gleichverteilt sind
- Peak Memory sinkt
- mehr partitionen => mehr Optimalitätsverlust, eventuell mit LP testen
- Heuristiken verbessern die Lösung
- Cut-Edge-Anzahl korreliert mit Quality Loss — Mehr Schnitte = mehr approximierte Constraints = schlechtere Lösung
- Complexity-Weighted Allocation Genauigkeit — Wie gut schätzt die Formel die variablen und wie gut die echte Solve-Zeit? => wahrscheinlich sehr schlecht
- Testen: Wie viel der optimalität wird abgeschnitten? - maybe einfach mit LP schauen (da das das optimale ist wie viel schlechter die decompositioned LP Lösung ist im vgl. zu None LP Lösung

9 Conclusion

Bibliography

[1] S. Chopra and P. Meindl, *Supply Chain Management: Strategy, Planning, and Operation*, 7th ed. Pearson, 2021.

[2] L. A. Wolsey, *Integer Programming*, 2nd ed. Wiley, 2020.

[3] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman, 1979.

[4] SAP SE, "SAP Integrated Business Planning." [Online]. Available: https://help.sap.com/docs/SAP_INTEGRATED_BUSINESS_PLANNING

[5] G. B. Dantzig and P. Wolfe, "Decomposition Principle for Linear Programs," *Operations Research*, vol. 8, no. 1, pp. 101–111, 1960.

[6] A. Schrijver, *Theory of Linear and Integer Programming*. Wiley, 1998.

[7] G. Barbarosoglu and D. Ozgur, "Hierarchical design of an integrated production and 2-echelon distribution system," *European Journal of Operational Research*, vol. 118, no. 3, pp. 464–484, 1999.

[8] H. Lidestam and M. Rönnqvist, "Use of Lagrangian decomposition in supply chain planning," *Mathematical and Computer Modelling*, vol. 54, no. 9–10, pp. 2428–2442, 2011.

[9] R. E. Bixby, "A Brief History of Linear and Mixed-Integer Programming Computation," *Documenta Mathematica*, vol. Extra Volume ISMP, pp. 107–121, 2012.

[10] L. Gurobi Optimization, "Gurobi Optimizer Reference Manual." [Online]. Available: https://www.gurobi.com/documentation/

[11] SCIP Optimization Suite, "SCIP: Solving Constraint Integer Programs." [Online]. Available: https://www.scipopt.org/

[12] U. Sandner, "Dekompositionsverfahren für die Optimierung der Supply Network-Planung," 1998.

[13] J. F. Shapiro, *Modeling the Supply Chain*, 2nd ed. Duxbury Press, 2006.

[14] J. F. Benders, "Partitioning Procedures for Solving Mixed-Variables Programming Problems," *Numerische Mathematik*, vol. 4, no. 1, pp. 238–252, 1962.

[15] M. L. Fisher, "The Lagrangian Relaxation Method for Solving Integer Programming Problems," *Management Science*, vol. 27, no. 1, pp. 1–18, 1981.

[16] A. M. Geoffrion, "Lagrangean Relaxation for Integer Programming," *Mathematical Programming Study*, vol. 2, pp. 82–114, 1974.

[17] H. I. Everett, "Generalized Lagrange Multiplier Method for Solving Problems of Optimum Allocation of Resources," *Operations Research*, vol. 11, no. 3, pp. 399–417, 1963.

[18] C. Lemaréchal, "Lagrangian Relaxation," in *Computational Combinatorial Optimization*, M. Jünger and D. Naddef, Eds., Springer, 2001, pp. 112–156.

[19] G. B. Dantzig and P. Wolfe, "Decomposition Principle for Linear Programs," *Operations Research*, vol. 8, no. 1, pp. 101–111, 1960.

[20] L. S. Lasdon, *Optimization Theory for Large Systems*. Macmillan, 1970.

[21] D. P. Bertsekas, *Nonlinear Programming*, 2nd ed. Athena Scientific, 1999.

[22] H. Stadtler, C. Kilger, and H. Meyr, *Supply Chain Management and Advanced Planning: Concepts, Models, Software, and Case Studies*, 5th ed. Springer, 2015.

[23] J. T. Dickersbach, *Supply Chain Management with SAP APO: Structures, Modelling Approaches and Implementation*, 3rd ed. Springer, 2009.

[24] G. B. Dantzig, *Linear Programming and Extensions*. Princeton University Press, 1963.

[25] L. Schewe, M. Schmidt, and D. Weninger, "A Decomposition Heuristic for Mixed-Integer Supply Chain Problems," *Operations Research Letters*, vol. 48, no. 3, pp. 236–244, 2020.

[26] H.-O. Günther and H. Meyr, *Supply Chain Planning: Quantitative Decision Support and Advanced Planning Solutions*. Springer, 2009.

[27] J. R. Jackson and I. E. Grossmann, "Temporal decomposition scheme for nonlinear multisite production planning and distribution models," *Industrial and Engineering Chemistry Research*, vol. 42, no. 13, pp. 3045–3055, 2003.

[28] K. Neumann and M. Morlock, *Operations Research*. Hanser, 1993.

[29] H. P. Williams, *Model Building in Mathematical Programming*, 3rd ed. John Wiley & Sons, 1991.

[30] A. J. Clark and H. Scarf, "Optimal Policies for a Multi-Echelon Inventory Problem," *Management Science*, vol. 6, no. 4, pp. 475–490, 1960.

[31] S. A. Bessler and A. F. Veinott Jr., "Optimal Policy for a Dynamic Multi-Echelon Inventory Model," *Naval Research Logistics Quarterly*, vol. 13, no. 4, pp. 355–389, 1966.

[32] P. Glasserman and S. Tayur, "The Stability of a Capacitated, Multi-Echelon Production-Inventory System Under a Base-Stock Policy," *Operations Research*, vol. 42, no. 5, pp. 913–925, 1994.

[33] F. Chen, "Optimal Policies for Multi-Echelon Inventory Problems with Batch Ordering," *Operations Research*, vol. 48, no. 3, pp. 376–389, 2000.

[34] E. B. Diks and A. G. de Kok, "Optimal Control of a Divergent Multi-Echelon Inventory System," *European Journal of Operational Research*, vol. 111, no. 1, pp. 75–97, 1998.

[35] K. Dogan and M. Goetschalckx, "A primal decomposition method for the integrated design of multi-period production-distribution systems," *IIE Transactions*, vol. 31, no. 11, pp. 1027–1036, 1999.

[36] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[37] V. D. R. Guide Jr. and L. N. Van Wassenhove, "Closed-Loop Supply Chains: A Critical Review, and Future Research," *Interfaces*, vol. 33, no. 6, pp. 57–66, 2003.

[38] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.

[39] Python Software Foundation, "Python Documentation." [Online]. Available: https://docs.python.org/3/

[40] NetworkX Developers, "NetworkX Documentation." [Online]. Available: https://networkx.org/documentation/stable/

[41] Polars Contributors, "Polars Documentation." [Online]. Available: https://docs.pola.rs/

[42] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

Index of Appendices

1 Appendix: Proof: Disjointness of Stage Sets ..... B
2 Appendix: Complexity Estimation Formulas ..... D
3 Appendix: Solver Configuration ..... F

1 Appendix: Proof: Disjointness of Stage Sets

Premise. Let $G_C = (V_C, E_C)$ be the condensation of the supply chain graph. By construction, G_C is a *DAG*.

Definitions. Let $P = \{v \in V_C : \tau(v) = \text{Prod}\}$ be the production nodes. Define $U_P = \left\{v \in V_C : \exists p \in P, v \xrightarrow{*} p\right\}$ (nodes with production downstream) and $D_P = \left\{v \in V_C : \exists p \in P, p \xrightarrow{*} v\right\}$ (nodes with production upstream), where $\xrightarrow{*}$ denotes reachability.

Lemma 1. The sets $U_P \setminus D_P$, $U_P \cap D_P$, $D_P \setminus U_P$, and $\overline{U_P \cup D_P}$ partition V_C.

Proof. For any universe U and sets $A, B \subseteq U$, the four sets $A \setminus B$, $A \cap B$, $B \setminus A$, and $U \setminus (A \cup B)$ are pairwise disjoint and their union equals U by definition. □

Theorem. The stage sets S_1, S_2, S_3, S_4, S_5 partition V_C.

Proof. Let $R = V_C \setminus (U_P \cup D_P)$, $R_1 = \{v \in R : \exists (u, v) \in E_C, \tau(u) = \text{Proc}\}$, and $R_5 = \{v \in R : \exists (v, w) \in E_C, \tau(w) = \text{Cust}\}$. We show (i) pairwise disjointness and (ii) completeness.

(i) Disjointness. By Lemma 1, the four base regions $U_P \setminus D_P$, $U_P \cap D_P$, $D_P \setminus U_P$, and R partition V_C. Each node belongs to exactly one base region and is assigned to a stage set based only on its region. Since regions are disjoint, we only need to show disjointness within each region.

Within $U_P \setminus D_P$: S_1 contains nodes that are procurement nodes or have procurement input, S_2 contains nodes that are neither procurement nodes nor have procurement input. These conditions are complementary, so $S_1 \cap S_2 = \emptyset$.

Within $U_P \cap D_P$: All nodes go to S_3.

Within $D_P \setminus U_P$: S_5 contains nodes that are customer nodes or have customer output, S_4 contains nodes that are neither customer nodes nor have customer output. These conditions are complementary, so $S_4 \cap S_5 = \emptyset$.

Within R: Nodes in R_1 go to S_1. Nodes in $R_5 \setminus R_1$ go to S_5. Nodes in $R \setminus R_1 \setminus R_5$ go to S_4. These three sets partition R by construction.

(ii) Completeness. By Lemma 1, every $v \in V_C$ belongs to exactly one base region:

- $U_P \setminus D_P$: Covered by $S_1 \cup S_2$ (procurement input or not).
- $U_P \cap D_P$: Equals S_3 by definition.
- $D_P \setminus U_P$: Covered by $S_4 \cup S_5$ (customer output or not).
- R: Covered by $R_1 \subset S_1$, $(R_5 \setminus R_1) \subset S_5$, and $(R \setminus R_1 \setminus R_5) \subset S_4$.

Thus $S_1 \cup S_2 \cup S_3 \cup S_4 \cup S_5 = V_C$. □

Corollary. The stage classification assigns each node to exactly one stage, ensuring no duplication or omission of nodes and therefore later variables and constraints during decomposition.

2 Appendix: Complexity Estimation Formulas

This appendix documents how subproblem complexity is estimated for runtime allocation. The estimates are computed from input data tables before the *MILP* model is built, counting expected variables based on entity attributes and horizon settings.

Several configurable horizons control when discrete variables are generated. Beyond each horizon, features revert to continuous variables or are disabled. Table 4 shows different forms of horizons.

Horizon	**Controls**
Discretization Horizon	When discrete lot sizes create integer execution variables
MinLot Horizon	When minimum lot sizes create binary indicator variables
FixCost Horizon	When fixed costs create binary setup indicator variables
FixCapa Horizon	When fixed capacity consumption creates binary indicator variables
Freeze Horizon	Periods with no new variables

Table 4: Horizon settings affecting variable estimation

Each horizon can be configured globally or per entity type (production, transport, customer transport). A value of -1 means unlimited (feature active for all periods).

A period is active and generates variables only if all conditions hold:

- the period is beyond the freeze horizon,
- falls within the entity's validity period,
- is not a calendar break,
- and has positive capacity.

Table 5 shows which attributes trigger discrete or continuous variables:

Entity Type	**Discrete Variables**	**Continuous Variables**
Production	+1 if lot size active within horizon +1 if min lot size active +1 if fixed cost or fixed capacity	+1 per active period without lot size
Transport	+1 if lot size active within horizon +1 if min lot size active +1 if fixed cost active	+1 per active period without lot size
Customer Transport	+1 if lot size active within horizon +1 if min lot size active +1 if fixed cost active	+1 per active period without lot size
External Procurement	+1 if fixed cost active	+1 per active period
Location Product	None	+1 stock variable per plannable period
Customer Product	None	+2 per demand period
Resources	None	None (constraints only)

Table 5: Variable estimation rules by entity type

When multiple features require a binary indicator, they share the same variable.

3 Appendix: Solver Configuration

add solver config